

ParallelKernelBench: Can LLMs Write Fast Multi-GPU Kernels?

Willy Chan^{1,2,†,*}, Nathan Paek^{1,2,†,*}, Simon Guo¹, Simran Arora^{3,2}, and Daniel Y. Fu^{4,2}

¹Stanford University

²Together AI

³California Institute of Technology

⁴University of California, San Diego

[†]Work done during internship at Together AI

^{*}Equal contribution

{willyc, nathanjp, simonguo}@stanford.edu, {simran, danfu}@together.ai

June 22, 2026

Abstract

There is growing interest in using large language models (LLMs) to write high-performance GPU kernels, with recent work showing promising results on single-GPU workloads. However, multi-GPU kernel generation remains unexplored, despite communication emerging as a dominant bottleneck in large-scale training and inference. In this paper, we study how well LLMs can write multi-GPU kernels, a task compounded by three challenges: (1) the design space is combinatorially large, as training and inference workloads can be parallelized across tensor, expert, pipeline, data, and sequence dimensions; (2) single-GPU memory-compute roofline analysis fails to capture communication bottlenecks in multi-GPU execution; and (3) the many hardware paths available for communication (e.g., copy engine, TMA, SM instructions) carry distinct tradeoffs. We introduce **ParallelKernelBench** (PKB), a benchmark and evaluation framework for multi-GPU kernel generation. To ensure that PKB achieves high coverage over the space of parallel AI workloads, we create a taxonomy of workloads spanning different parallelism types and select 87 problems covering these. We contribute evaluations of frontier coding models on PKB, finding that LLMs struggle: zero-shot, the best model gets only 32% of PKB correct with only 25% being faster than the unoverlapped baseline (PyTorch + NCCL). We contribute a communication-aware roofline to analyze correct kernels, finding that over 90% of baselines achieve less than 50% of peak hardware utilization. Optimized solutions to many PKB workloads yield net new useful kernels: we highlight several LLM-generated net new kernels that outperform their reference implementations in specific regimes, including NeMo’s vocab-parallel log-probability kernel (up to 1.28×), Hyena CP (up to 3.55×), and SAM3 IoU suppression (up to 1.38×). Code and benchmark are available at github.com/togethercomputer/ParallelKernelBench.

1 Introduction

In recent years, large language models (LLMs) have made rapid progress on GPU kernel generation [6, 30]. Several benchmarks have emerged to track and drive this progress, establishing standardized evaluation frameworks that guide model development to optimize single-GPU workloads [32, 41, 54]. As a result, the majority of efforts to automate kernel generation, including agentic scaffolding [13], model finetuning [6], and emerging industry tools [28] have focused on the single-GPU regime. However, modern AI workloads increasingly require kernels that span multiple GPUs as model

sizes, sequence lengths, and serving throughput requirements exceed the limits of a single GPU. We provide the first standardized benchmark to understand model capabilities in this setting.

Frontier LLM architectures have outgrown the memory and compute of a single GPU [4]. On many production distributed training and inference workloads, communication increasingly consumes the majority of runtime and yields low Model FLOPs Utilization at scale [15, 16, 26]; for Llama-3.3-70B [20], all-reduce alone accounts for over 20% of end-to-end inference latency on a DGX H100, even with NVLink and in-switch reductions [19]. Hardware evolution has widened this gap: FP16 tensor core throughput on Streaming Multiprocessors (SMs) has grown over $12\times$ while NVLink bandwidth has grown only $6\times$ from NVIDIA Ampere [29] to Rubin [36], and scale-up domains such as NVL144 and NVL576 continue to add complexity [7]. Furthermore, while AI accelerators have converged on technologies like HBM and systolic arrays for GEMMs, the networking landscape remains heterogeneous—NVIDIA uses NVLink, AMD uses xGMI [45], TPUs use Inter-Chip Interconnect [27]—with new topologies still emerging. This diversity makes a parallel kernel benchmark all the more valuable and timely.

Evaluating LLMs on multi-GPU kernel generation, however, presents three distinct challenges:

1. The **problem design space** expands combinatorially beyond the single-GPU case. Practitioners compose parallelism strategies (e.g. Tensor, Expert, Data, Context, and Sequence Parallelism) according to hardware constraints [20, 37], with each composition inducing a different communication pattern. This results in a vast space of problems that a benchmark must cover, and it is unclear which combinations of these parallelism strategies appear in real AI workloads.
2. Existing tools for measuring the hardware efficiency of AI kernels are **not communication-aware**. Standard evaluation often compares a kernel against the compute and memory-bandwidth limits predicted by the standard Roofline model [12, 52]. In distributed GPU programs, however, the relevant bottleneck might include interconnect bandwidth, collective latency, or taxonomy-dependent communication rather than local compute or memory bandwidth. Thus, in the distributed setting, the performance target must be derived per workload from the expected compute, memory traffic, communication volume, collective pattern, and hardware topology [9, 11].
3. While single-GPU kernels already involve optimization choices over tiling, parallelism, memory hierarchy usage, and software pipelining, multi-GPU kernels add a new axis of **communication mechanism selection**. Communication can be placed intra-SM (fused into the same kernel as compute) or inter-SM (in a separate kernel), and launched via different hardware mechanisms (e.g., copy engine, TMA, SM load/store, NVLS), each with distinct performance tradeoffs [48].

To address these challenges, we introduce **ParallelKernelBench** (PKB), a benchmark designed to generate and evaluate multi-GPU kernels. PKB provides 87 problems organized under a taxonomy that spans different parallelism strategies and distills core communication patterns that arise in real AI workloads, from inference to RL to post-training. The benchmark takes a PyTorch and NCCL reference as input and challenges models to produce an optimized CUDA kernel, directly exposing model capabilities on low-level distributed optimization. To evaluate and analyze generated kernels, we propose a communication-aware roofline model that extends the standard roofline with interconnect bandwidth: situating both baseline and generated implementations against hardware limits across *compute*, *memory*, and *communication* bottlenecks.

We evaluate frontier coding models (GPT-5.5, Opus-4.7, Gemini 3 Pro, GLM-5.2, DeepSeek V4 Pro) on correctness and performance. Results show that multi-GPU kernel generation remains

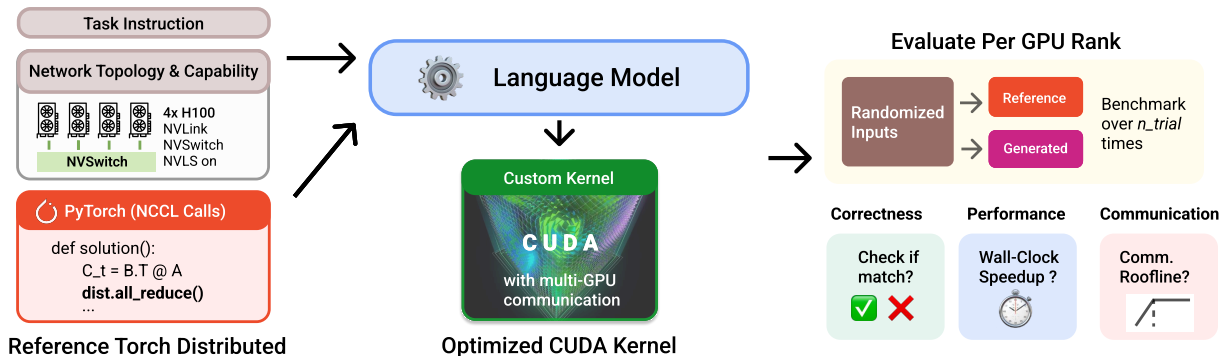


Figure 1: **ParallelKernelBench evaluates language models on multi-GPU kernel optimization.** Given a system topology and a baseline PyTorch/NCCL reference implementation, models must rewrite the program as a custom distributed CUDA kernel. The reference reflects the standard communication stack used in modern distributed training frameworks, while the target replaces coarse-grained library collectives with fine-grained CUDA kernels that use Unified Virtual Addressing (UVA) for direct peer-to-peer transfers over NVLink.

difficult: GPT-5.5 produces correct kernels for fewer than 33% of problems, and only 25% match or exceed the unoverlapped Torch + NCCL baseline (fast_1). Failures stem from challenging distributed CUDA syntax, complex parallelization reasoning, and limited use of advanced communication mechanisms. An agentic harness with execution feedback [55] improves GPT-5.5 correctness to 36.7% and fast_1 to 29.8%, but substantial headroom remains. Open-source models underperform significantly, despite recent progress on single-GPU benchmarks such as KernelBench Level 3 [18]. Nevertheless, generated solutions can produce *net-new* high-performance kernels for certain input tensor sizes, with speedups on NeMo vocab-parallel log-probability (up to $1.28\times$), Ulysses attention (up to $3.73\times$), Hyena CP (up to $3.55\times$), and SAM3 IoU suppression (up to $1.38\times$); see Appendix F.

These findings highlight the gap between current model capabilities and fully autonomous LLM-based optimization of multi-GPU workloads. We release PKB as an open benchmark to drive community progress toward automated distributed GPU code generation available at github.com/togethercomputer/ParallelKernelBench. Concretely, we contribute:

1. A **taxonomy** for describing distributed AI workloads in terms of parallelism strategies and sharding configurations, used to organize and curate PKB’s problem set.
2. ParallelKernelBench, a **benchmark** of 87 diverse, real-world, and *net new* multi-GPU problems paired with a communication-aware evaluation framework that situates kernel performance against compute, memory, and communication ceilings.
3. **Analysis** of kernels generated by frontier LLMs, revealing that models systematically underperform by defaulting to simple communication primitives rather than more complex, higher-performing alternatives.

2 Background & Related Work

In this section, we provide background on modern GPU networking hardware and software, as well as a review of prior work (Table 1) related to LLM-driven kernel generation.

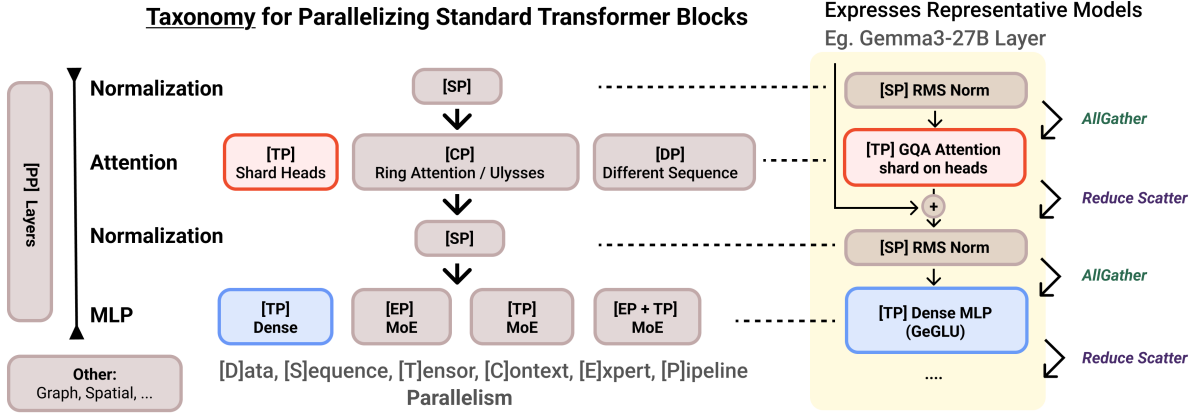


Figure 2: **A taxonomy of parallelism strategies for standard transformer workloads.** A standard transformer layer (norm $\rightarrow$ attention $\rightarrow$ norm $\rightarrow$ MLP) can be parallelized across data, sequence, tensor, context, layer/pipeline, and expert dimensions. We provide an instantiation of this taxonomy as an example on a Gemma3-27B layer, annotating each operation with its corresponding NCCL collective. This taxonomy guides the construction of PKB problems.

2.1 Background

GPU Architecture. Modern datacenter-grade NVIDIA GPUs execute programs called *kernels* across thousands of parallel hardware threads organized into a three-level hierarchy: *threads* are the basic unit of execution; threads are grouped into *warps* of 32 that execute on a single SM; and warps are grouped into *blocks* that share a fast on-chip shared memory scratchpad. GPUs also expose high-bandwidth memory (HBM) as the primary large working set memory. On multi-GPU systems, GPUs are connected via *NVLink*, an interconnect that allows threads executing on one GPU to directly read from and write to the HBM of a peer GPU.

Interconnect Hardware. Modern GPU clusters expose a hierarchy of interconnects. PCIe (64 GB/s) handles CPU-to-GPU transfers, while NVLink (450 GB/s unidirectional per GPU) provides high-bandwidth point-to-point GPU-to-GPU connectivity within a node. NVSwitch connects all GPUs within a node in a full all-to-all fabric and supports in-network compute via NVLink SHARP (NVLS)—exposed through *multimem* instructions—accelerating multicast and reduction operations without routing data through GPU SMs. Unless otherwise noted, all inter-GPU communication in this paper occurs over NVLink/NVSwitch.

Transfer Mechanisms. Following the taxonomy of ParallelKittens [48], GPU-to-GPU data transfer occurs via three primary mechanisms. The *copy engine* is host-initiated and saturates NVLink bandwidth for large messages, but cannot be invoked from within a kernel. *Tensor Memory Accelerator (TMA)* transfers move data asynchronously without consuming SM resources, making them well-suited for overlapping communication with compute. Finally, *SM load/store (LSA)* instructions perform transfers directly using SMs; while this increases register pressure, it enables NVLS in-switch compute via NVSwitch, which the other two mechanisms do not support.

Communication Software. NCCL is the standard communication library for GPU collectives and is used by `torch.distributed`: exposing operations such as all-reduce and reduce-scatter as asynchronous kernels enqueued on a CUDA stream [22]. While convenient, NCCL typically operates at the granularity of full collectives with limited flexibility for fine-grained compute-communication overlap. For lower-level control, programmers can use a *symmetric heap* - a memory region registered

across all participating GPUs - to perform direct one-sided puts and gets from within a kernel [44]. PyTorch exposes this via its symmetric memory abstraction, accessible from CUDA through Unified Virtual Addressing (UVA), which maps remote GPU memory into a single virtual address space so that kernels can read from and write to peer GPUs using standard load/store instructions.

2.2 Related Work

Benchmark	# Problems	Multi-GPU?	Workload	Sources
KernelBench	250	×	Inference	PyTorch Primitives HuggingFace Models
FlashInfer-Bench	18	×	Inference	Serving Traces
SOL-ExecBench	235	×	Inference	Model Subgraphs
MultiKernelBench	285	×	Training, Inference	KernelBench Extensions
ParallelKernelBench	87	✓	Training, Inference	Production Frameworks & Libraries

Table 1: **Existing benchmarks are limited in multi-GPU problem coverage.** Online sources contain no distributed problems and do not cover the full breadth of workloads.

Custom Distributed Kernels. Many specialized libraries exist to optimize specific AI operations by overlapping communication with computation, such as Flux [33], DeepEP [14], and FlashMoE [3]. They employ fine-grained device-side communication operations to yield substantial latency and throughput gains over naive unoverlapped implementations. However, they are manually tuned and do not generalize between hardware generations, motivating automated kernel generation.

LLM-Based Kernel Generation. KernelBench [41] established the standard benchmark for LLM-generated GPU kernels, with subsequent work (e.g. FlashInfer-Bench [54], SOL-ExecBench [32], MultiKernelBench [51]) expanding coverage to new problem types and hardware platforms. This has spurred optimization efforts including evolutionary search [30] and multi turn-based RL [6] to improve kernel quality. However, all of these works operate exclusively in the single-GPU setting. The closest multi-GPU effort is CUCo [21], an agentic framework for joint computation-communication kernel generation, but it evaluates only four workloads and restricts to the transformation of replacing host-initiated NCCL collectives with device-initiated NCCL GIN calls.

Multi-GPU Kernel DSLs. Several frameworks lower the barrier to writing multi-GPU kernels, such as ParallelKittens [48], Triton-Distributed [61], Iris [5], and PyTorch Symmetric Memory [58]. However, all still require expert human implementation. For a discussion of device-side communication primitives and a survey of modern kernel-writing frameworks, see Appendix B.

3 ParallelKernelBench

In this section, we describe ParallelKernelBench: its task format, the principles guiding problem selection, and its evaluation metrics.

3.1 ParallelKernelBench Task Format

Each benchmark task presents the model with an unoptimized reference implementation written in PyTorch with `torch.distributed` NCCL operations, along with a system topology specifying the number of ranks and intra-node hardware configuration. The model is tasked with rewriting this reference into a performant CUDA kernel that uses unified virtual addressing. This is performed via PyTorch’s symmetric memory API, an interface for low-level peer-to-peer GPU communication in PyTorch. The end-to-end generation and evaluation cycle is illustrated in Figure 1 and the full prompt template is provided in Appendix A.

Prerequisites. Each task defines a case in the common `create_input_tensor()` function that initializes per-rank input tensors and sizes, and pre-configures all communication groups. The sharding scheme is fully determined before the model’s `solution()` function is invoked, which runs identically on every rank. Note that depending on the task, per-rank inputs may differ. One minimal in-context example is provided: a distributed add kernel demonstrating symmetric memory setup/pointer caching and NVLink P2P data transfer via pointers.

Task input. For each task, the model is provided: (i) a reference implementation using unoptimized `torch.distributed` and NCCL collectives; (ii) a system prompt and in-context example; and (iii) a natural language system topology description specifying hardware capabilities such as NVLink P2P access and in-switch reduction support.

Task output. The model produces a lower-level rewrite of the reference `solution` function that replaces the unoptimized collective with a fine-grained kernel that overlaps computation and communication. The primary evaluation uses **CUDA** with **Torch Symmetric Memory** via PyTorch `load_inline`. Symmetric heap pointers are cached across trials so that setup overhead is excluded from measured latency.

3.2 Problem Selection

PKB problems are drawn from two signals of practical relevance: GitHub repositories scraped to identify operations frequent in real deployments, and optimized library implementations as evidence that the community considers a workload worth optimizing. Together, these sources ensure PKB reflects what practitioners encounter in production. We further characterize standard transformer models from these sources (Figure 2), decomposing each layer into parallelizable operations and annotating the required NCCL collectives; this serves as a systematic guide for constructing PKB problems with coverage across parallelism strategies and collective types. See Figure 3 for a survey of PKB’s data sources and Appendix C for source links.

4 Understanding the Baseline

We describe the metrics used to evaluate LLM-generated kernels and show that correct baseline solutions remain far from their theoretical hardware limits, establishing that high-performance solutions to PKB problems would be genuinely net new.

4.1 Metric Design

When comparing across models, we report *fast_p* as introduced in KernelBench [41], which measures the proportion of tasks for which a model produces a kernel that is both correct and faster than

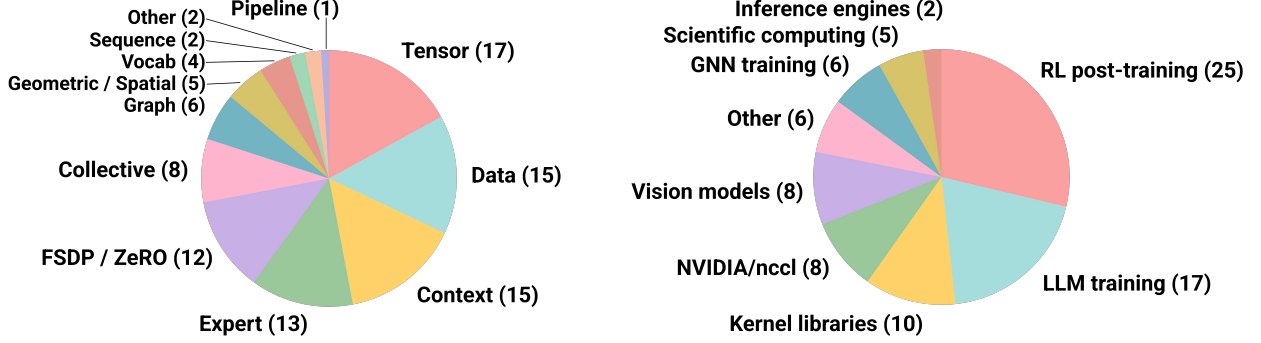


Figure 3: **ParallelKernelBench** covers a diverse set of distributed GPU workloads. Problems span the major parallelism strategies used in production and come from industry codebases.

the baseline. This provides a single scalar summary of model capability that jointly penalizes both incorrectness and failure to improve over the naive implementation. We also evaluate kernels along correctness, performance, and hardware utilization.

Correctness. For each task, we run 5 trials with random per-rank PyTorch inputs and verify that the output of the generated `solution()` matches that of the reference within numerical tolerance.

Performance. We measure wall-clock execution time averaged over 100 profiling iterations, preceded by 500 warmup iterations to reach a power-steady state, with kernels launched back-to-back without intermediate synchronization [47]. Generated kernels are compared against the unoverlapped PyTorch + NCCL reference baseline.

4.2 Understanding the Communication Bottleneck

We introduce roof_u , the fraction of tasks for which the generated solution is both correct and achieves at least fraction u of the hardware-limited roofline bound. We contribute the first systematic application of roofline analysis as an automatic evaluation metric over a diverse distributed kernel benchmark; while fast_p asks whether a solution beats the PyTorch baseline, roof_u asks whether it uses the hardware well — measuring utilization relative to whichever resource (compute, memory, or communication) is the actual bottleneck.

Concretely, for each task i we contribute a theoretical cost model (Appendix D) that derives roofline performance estimates from FLOP and byte counts, modeling the fastest achievable runtime under perfect overlap of compute, memory, and communication, i.e. the runtime if the solution were limited only by the binding hardware resource [11].

$$T_{\text{roof},i} := \max \left(\frac{\text{FLOPs}_i}{\text{Peak Compute Throughput}}, \frac{\text{Bytes Moved From Memory}_i}{\text{Peak Memory Bandwidth}}, \frac{\text{Bytes Communicated}_i}{\text{Peak NVLink Bandwidth}} \right)$$

Roofline utilization is then $U_i = \min(1, \text{Time}_{\text{roof},i} / \text{Time}_{\text{sol},i})$, and

$$\text{roof}_u := \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{correct}_i \wedge U_i > u].$$

Baseline programs underutilize hardware. Before evaluating LLM-generated kernels, we first ask whether the naive PyTorch + NCCL reference leaves meaningful headroom — if baselines already approach hardware limits, there is little to optimize. Figure 4 shows the opposite: the majority of PyTorch baselines fall below 50% of their communication-aware roofline bound. See Appendix Table 6 for a per-bottleneck breakdown showing this gap is starkest on communication-bound tasks. With more fine-grained operations, LLM-generated solutions can potentially close this gap.

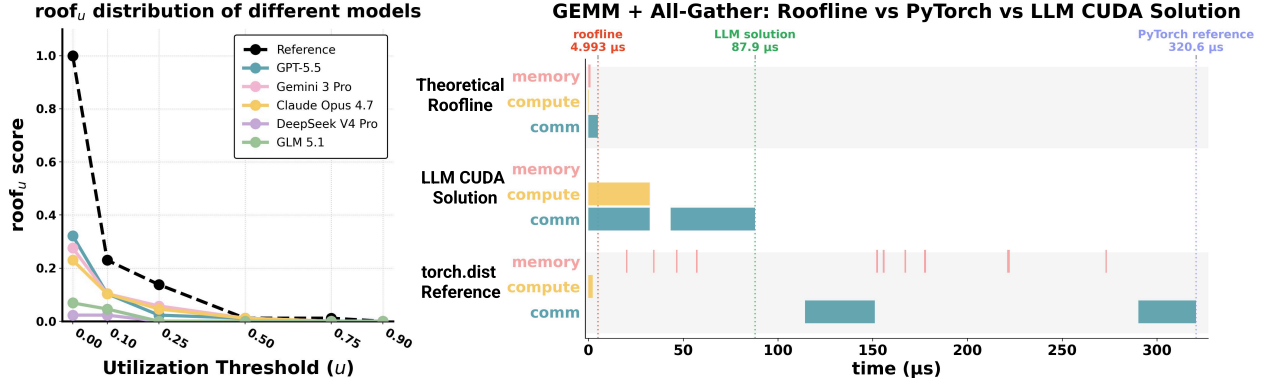


Figure 4: **PyTorch baselines fall far below the roofline; LLM solutions can close the gap.** (Left) Distribution of $roof_u$ scores shows both reference and generated kernels substantially underperform in hardware utilization. (Right) Profiler trace for problem 14_gemm_all_gather: the PyTorch reference launches GEMM, NCCL all-gather, and a concat copy as separate kernels with stream-sync overhead between them. The optimized solution fuses GEMM and gather into a single WMMA kernel that writes output tiles directly to every peer’s symmetric buffer over NVLink, decreasing runtime toward the theoretical minimum. Gaps are largely CPU scheduling latency.

5 Model Evaluation

Models are evaluated via a single-attempt prompt containing an in-context example covering key boilerplate code, such as symmetric memory allocation, P2P data transfer, and using the `multimem NVLS` instruction (see Appendix A). All experiments were conducted on a single node with 4 NVIDIA H100 80GB GPUs interconnected via 4th-generation NVLink/NVSwitch running PyTorch 2.11 and CUDA 12.6. All inputs use BF16 precision.

While the experiments in this section use Hopper, the PKB generation and evaluation framework also supports Blackwell, and we report results in Appendix G.

5.1 Correctness & Error Analysis

Table 2 shows that correct solutions concentrate in collective primitives, TP-sharded GEMMs, and Ulysses context-parallel problems across all models. We hypothesize this reflects training data bias: these patterns are heavily represented in open-source multi-GPU frameworks [47, 48], and optimized reference implementations are widely available online [22, 25, 48], giving models stronger priors.

Figure 5 classifies failure modes across all problems. Open-source and non-reasoning models fail predominantly at the syntax level — compiler and interpreter errors indicate they lack fluency with CUDA and Torch APIs. Reasoning models largely overcome this, with compilation rates exceeding 50%, yet correctness remains low; the majority of kernels produce incorrect outputs, suggesting models cannot reliably reason about rank coordination, data partitioning, and collective ordering.

Category	#	GPT-5.5		Claude Opus 4.7		Gemini 3 Pro		GLM-5.2		DeepSeek V4 Pro	
		pass	fast ₁	pass	fast ₁	pass	fast ₁	pass	fast ₁	pass	fast ₁
		@1→3	@1→3	@1→3	@1→3	@1→3	@1→3	@1→3	@1→3	@1→3	@1→3
Collective Primitive	8	3 → 5	3 → 4	4 → 5	3 → 4	6 → 6	2 → 2	1 → 3	1 → 1	0 → 0	0 → 0
Tensor Parallel	17	2 → 2	1 → 2	1 → 3	1 → 3	3 → 3	1 → 1	0 → 1	0 → 0	0 → 0	0 → 0
Sequence Parallel	2	1 → 1	1 → 1	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0
Context Parallel	12	7 → 8	5 → 6	7 → 7	3 → 4	7 → 9	5 → 7	2 → 2	0 → 0	2 → 3	0 → 1
Pipeline Parallel	1	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0	0 → 0
Data Parallel	9	1 → 2	1 → 2	1 → 1	1 → 1	0 → 1	0 → 1	0 → 0	0 → 0	0 → 0	0 → 0
Expert Parallel	11	3 → 3	2 → 3	2 → 6	0 → 1	3 → 4	0 → 1	1 → 1	0 → 0	0 → 0	0 → 0
FSDP / ZeRO	9	3 → 4	3 → 4	3 → 3	3 → 3	1 → 1	0 → 1	0 → 0	0 → 0	0 → 0	0 → 0
Vocab Parallel	4	2 → 2	1 → 1	1 → 2	1 → 2	2 → 2	2 → 2	0 → 1	0 → 1	0 → 0	0 → 0
Graph Parallel	6	2 → 4	2 → 3	0 → 2	0 → 2	1 → 1	1 → 1	0 → 0	0 → 0	0 → 0	0 → 0
Geometric / Spatial	5	3 → 3	3 → 3	1 → 2	0 → 0	0 → 1	0 → 1	0 → 0	0 → 0	0 → 0	0 → 0
Other	2	1 → 2	0 → 1	0 → 0	0 → 0	1 → 2	1 → 2	0 → 1	0 → 0	0 → 0	0 → 0
Total	87	28 → 36	22 → 27	20 → 31	12 → 20	24 → 30	12 → 19	4 → 9	1 → 2	2 → 3	0 → 1

Table 2: **Single-shot LLMs struggle on PKB.** pass@k reports correctness on the best of k attempts; fast₁@k counts solutions that are both correct and outperform the PyTorch + NCCL baseline. Repeated sampling improves both metrics, but fast₁@3 still peaks at just 31% (GPT-5.5), suggesting fundamental limitations beyond sampling noise.

The shift from syntax failures to correctness failures across model families reveals that the core bottleneck is not code generation fluency but a deeper failure to reason about parallel execution.

5.2 Speedup Distribution

As shown in Figure 6, fewer than 30% of one-shot LLM-generated kernels are both correct and faster than the PyTorch + NCCL reference. Figure 4 highlights how, once correctness is established, speedups naturally arise from eliminating NCCL staging overhead in favor of direct NVLink loads and stores. Analyzing the SASS assembly of correctly compiled kernels (Figure 5), we find that the vast majority rely on copy engine and SM load/store instructions for inter-GPU transfers, while more advanced mechanisms — TMA, `multimem`, and intra-SM warp specialization — are nearly absent. This suggests that one-shot models gravitate toward simple transfer primitives and have not learned to compose fine-grained overlap techniques that unlock peak hardware performance.

5.3 Impact of Agentic Harness & Iterative Feedback

Having established that base models struggle significantly on ParallelKernelBench, we now investigate whether iterative feedback from an agentic harness can improve performance.

Setup. For the agentic setting, we wrap our generation pipeline in a multi-turn harness built on mini-swe-agent [55], giving one of the best-performing base models (Gemini 3 Pro) access to a local bash environment rooted at the repository. Building on the one-shot prompt, the agent can compile, evaluate for correctness and speedup via a provided script, and iteratively refine its kernel via a local terminal environment. For more details on the harness, see Appendix E.

Performance. Figure 7 shows that iterative feedback allows the agent to hill-climb beyond single-shot performance, though gains quickly plateau—revealing a ceiling imposed by the model’s capacity to reason about distributed kernel optimization. The agent solved 35 of 87 problems (40.22%), up from 24 in the single-shot baseline, with 26 achieving speedup $> 1\times$ over the reference.

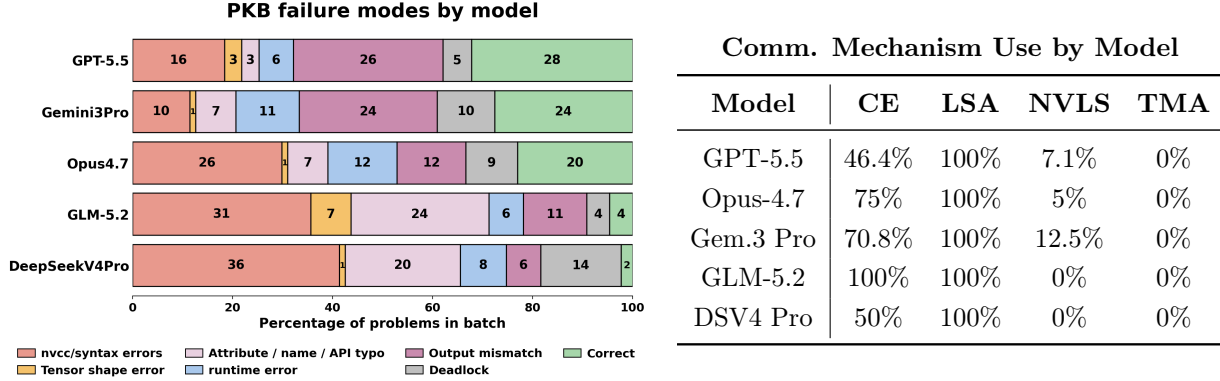


Figure 5: **Single-shot LLMs struggle to parallelize computation correctly and default to simplistic inter-GPU transfer mechanisms.** Reasoning models reduce syntax errors, but correctness remains low, revealing a fundamental gap in understanding multi-GPU distributed memory and rank coordination. CE (copy engine) and LSA (SM load/store) dominate generated kernels; NVLS/multimem and TMA instructions for low-overhead data movement are nearly absent.

6 Qualitative Kernel Analysis

In this section, we highlight common themes among LLM-generated kernels that achieved notable speedups over the PyTorch baseline, including several net-new kernels with no existing optimized open-source implementation, and discuss the broader implications. Detailed kernel analyses are provided in Appendix F.

6.1 Common LLM Optimizations

NVLink P2P Data Accesses. The most consistent source of speedup across problems comes from eliminating collective staging and packing overhead entirely in favor of direct peer memory access via SM load/store instructions, transparently routed through NVSwitch. Rather than paying for NCCL’s staging overhead, kernels read remote ranks’ symmetric memory buffers as pointer dereferences, fusing communication directly into the kernel. An example of this behavior is provided in Figure 8.

Movement Efficiency. Avoiding slow networked transfers—or batching them into as few wide chunks as possible—reduces execution time. LLMs generated kernels for several collective operations that matched or surpassed NCCL at small (1024×1024) tensor sizes (e.g., AllGather at $1.2\times$, Scatter at $1.6\times$) by dynamically selecting the widest available memory instructions based on alignment, with certain MoE and Ulysses kernels further exploiting a “fast path” of 128-bit aligned vectorized loads across NVLink.

Pipelining. Once NVLink bandwidth is the bottleneck, the goal is to hide latency with computation - communication overlap. In the generated kernels, we found examples (e.g. Ulysses All-to-All ($1.84\times$)) of the LLM attempting this using separate CUDA streams: each data chunk is processed on its own CUDA stream, allowing compute on one chunk to overlap with NVLink transfers for the next.

6.2 Net New Kernels

Beyond speedups on standard collectives, single-shot LLM generation occasionally uncovers *net new* high-performance kernels for workloads lacking optimized public references. Crucially, the models can

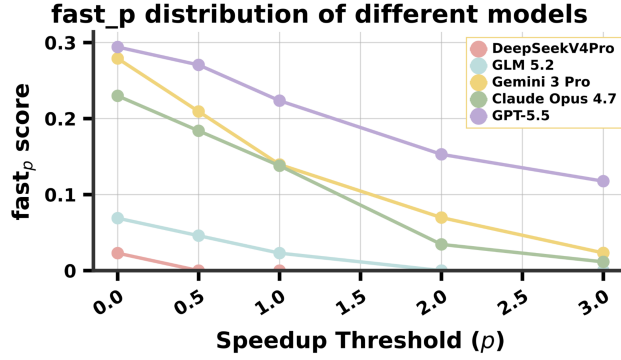


Figure 6: **Single-shot kernels rarely achieve meaningful speedup.** fast_p (the percent of correct kernels exceeding $p \times$ speedup) drops sharply with p . Models exceed the unoverlapped baseline in fewer than 30% of cases.

Agent Progress over Iterations (Gemini 3 Pro)

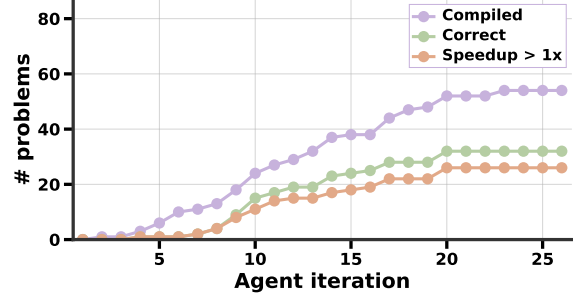


Figure 7: **Test-time compute enables hill-climbing on PKB.** We provide Gemini 3 Pro with a Mini-SWE-Agent harness and terminal environment, allowing it to make arbitrary changes to a codebase. Achieves 7 more correct (31) and 14 more fast_1 (26) solutions than single-shot.

PyTorch Reference:

```

1 def ulysses_allgather_reference():
2     x = x.contiguous()
3     x_size = torch.tensor(x.size())
4     # [1] Gather tensor sizes
5     size_list = [torch.zeros(...) for ...]
6     dist.all_gather(size_list, x_size)
7     # [2] Gather tensor data
8     tensor_list = [torch.empty(...) ...]
9     dist.all_gather(
10         tensor_list, x, group=group)

```

LLM-Generated Solution:

```

1 // Replaces [1] + [2]: single kernel
2 // with direct NVLink loads
3 template<typename T>
4 __global__ void
5 ulysses_allgather_kernel(...):
6     int64_t src_idx = ...
7     int64_t dst_idx = ...
8     // One-sided peer memory read
9     out[dst_idx] = src[src_idx];
10
11 ulysses_allgather_kernel
12 <<<blocks, threads, 0>>>(...)

```

Figure 8: **LLM eliminates NCCL staging overhead in Ulysses all-gather via direct NVLink loads.** The reference implementation uses two `dist.all_gather` calls routed through NCCL’s staging buffers: one to exchange tensor sizes and one for the data itself. The LLM-generated solution replaces both with a single CUDA kernel that reads directly from peer GPU memory over NVLink, bypassing NCCL entirely and eliminating the intermediate copy.

demonstrate strong performance in non-Transformer domains—such as state-space models, genomic pipelines, and multimodal RL loops. While human experts heavily optimize standard LLM workloads [32, 41, 48], such specialized kernels remain neglected. We highlight three such emergent kernels in Table 3, each replacing NCCL collectives with fused symmetric-memory kernels that exploit direct peer access over NVLink. Detailed analysis and tensor sweeps are provided in Appendix F.

NeMo Vocab-Parallel LogProb with Top- k /Top- p Filtering (Gemini 3 Pro). This kernel computes vocabulary-sharded log-probabilities under a top- k /top- p filtered distribution, which is a core part of NVIDIA NeMo-RL’s GRPO training loop [1]. While the baseline implementation requires an `all_to_all` collective to gather the full vocabulary for filtering followed by a final `all_gather`, the optimized solution completely bypasses NCCL collectives. It replaces them with a custom 128-bit symmetric memory push kernel that permutes the layout inline via index arithmetic, while fusing the log-softmax, token extraction, and target gather operations into a single block-per-token warp-shuffle reduction. As a result, the kernel achieves up to a $1.28\times$ speedup on the baseline configuration, with

Problem	Description	Sequence of Operations	Optimization Mode	Speedup
SAM3 Mask IoU Detection & Suppression	Cross-rank occlusion tracking for multi-object video segmentation.	AllGather → Overlap Detection → Frame Logic → Masking	• Fuses dual variable all-gathers into a single barrier. • Fuses UVA peer reads with 32× bitpacking & warp <code>__popc</code> area reductions. • Replaces dense float matrix math with hardware bitwise logic.	1.38× (<code>num_objs</code> = 16)
Hyena Forward (Context Parallel)	Context-parallel forward pass for Hyena models.	AlltoAll → Gate/FFT/Bias → AlltoAll	• Collapses 3 sequential <code>all_to_all_single</code> collectives into 1 rendezvous buffer. • Pulls remote slices over NVLink via <code>gather_x1_and_u</code> kernel. • Executes gating ($x_2 \odot v$) and inverse zigzag reindexing inline.	3.55× (<code>local_seq</code> = 1024)
NeMo Vocab LogProb with Top- k	Vocabulary-parallel log probabilities & filtering for alignment/RL.	Vocab-Gather → LogProb Compute → Top- k Masking	• Replaces <code>all_to_all_single</code> with a 128-bit (<code>uint4</code>) symmetric memory push kernel. • Permutes layout inline via index arithmetic. • Fuses log-softmax, token extraction, and all-gather via warp-shuffle reductions.	1.28× (<code>local_seq</code> = 1024)

Table 3: **Evaluation of LLM-generated symmetric-memory kernels on under-optimized non-Transformer workloads.** The single-shot models exploit direct NVLink peer access to eliminate heavy NCCL collectives and fuse compute-communication pipelines.

high performance solidly sustained across all evaluated sequence lengths (Figure 14).

Hyena Forward CP (GPT-5.5). This kernel implements the context-parallel forward pass of the Hyena operator [43]. Because the core Fast Fourier Transform (FFT) convolution requires sequence-global context, the reference implementation alternates between sequence-sharded and channel-sharded layouts using consecutive `all_to_all` communication round-trips. The optimized solution instead leverages direct NVLink peer access to bypass these collectives entirely. It packs the inputs into a single symmetric memory allocation and streams remote slices into a unified kernel that computes the gating operations ($x_2 \odot v$) and inverse zigzag reindexing inline. While this approach yields a peak speedup of $\sim 3.55\times$ at small working sets ($l = 1024$ for local length l) by eliminating framework and collective launch overheads, performance falls to a penalty of $\sim 0.70\times$ at large sequence lengths ($l \geq 8192$) where highly optimized bulk NCCL transfers saturate cross-GPU bandwidth more effectively than fine-grained UVA thread accesses (Figure 16).

SAM3 All-Gathered Mask IoU Suppression (GPT-5.5). This kernel resolves multi-GPU object conflicts in the SAM 3 video segmentation model [10] by performing global cross-rank intersection-over-union (IoU) checks to suppress duplicate region predictions after each frame. While the reference implementation relies on two variable-length `all_gather` collectives followed by a dense floating-point matrix multiplication to evaluate the IoU matrix, the optimized solution completely bypasses NCCL. It collapses the pipeline into four symmetric-memory kernels: it packs local masks and scores into a single shared symmetric buffer, streams peer masks over UVA pointers while bitpacking them into 32-bit words, computes pairwise intersections using fast bitwise hardware logic (`__popc`), and writes no-object logits directly to suppressed masks inline. This custom symmetric-memory strategy slashes framework launch and allocation overheads, achieving a peak speedup of $\sim 1.38\times$ for 16 tracked objects at a fixed 256×256 mask resolution. As the workload scales up to 512 objects (Figure 18), the quadratic scaling of the core pairwise comparison logic eventually dominates the runtime profile, causing the relative speedup to plateau at $\sim 1.03\times$ over the reference.

Takeaway. Across these non-Transformer workloads, the optimized kernels consistently replace high-overhead NCCL collectives with direct, packed symmetric-memory access, fuse downstream computation directly into the peer data-movement passes, and chunk sequences to overlap synchronization barriers. These implementations reveal a nuanced architectural trade-off: leveraging fine-grained UVA peer access slashes framework launch and allocation overheads to achieve speedups at small-to-medium working sets. However, as data volume scales, this specialized pattern cedes its advantage to bulk NCCL transfers, which saturate cross-GPU bandwidth more effectively.

7 Conclusion

We present three contributions: (1) an organization of distributed AI workloads by parallelism strategy; (2) ParallelKernelBench, a benchmark of 87 multi-GPU problems evaluated against compute, memory, and interconnect ceilings; (3) an analysis of frontier LLM-generated kernels, revealing where models succeed and fall short in communication-aware optimization. Crucially, all baselines use standard PyTorch and NCCL collectives, ensuring that they remain valid as new hardware topologies emerge. For these reasons, PKB provides a concrete target toward the longer-term goal of LLM systems that can autonomously optimize and manage large-scale distributed infrastructure. We provide limitations and directions for future work in Appendix H.

8 Acknowledgements

We are grateful to Together AI for supporting this work. We thank Stuart Sul, William Hu and Omar Abul-Hassan for insightful discussions and constructive feedback in shaping this work.

References

- [1] Nemo rl: A scalable and efficient post-training library. <https://github.com/NVIDIA-NeMo/RL>, 2025. GitHub repository.
- [2] Sand. ai, Hansi Teng, Hongyu Jia, Lei Sun, Lingzhi Li, Maolin Li, Mingqiu Tang, Shuai Han, Tianning Zhang, W. Q. Zhang, Weifeng Luo, Xiaoyang Kang, Yuchen Sun, Yue Cao, Yunpeng Huang, Yutong Lin, Yuxin Fang, Zewei Tao, Zheng Zhang, Zhongshu Wang, Zixun Liu, Dai Shi, Guoli Su, Hanwen Sun, Hong Pan, Jie Wang, Jiexin Sheng, Min Cui, Min Hu, Ming Yan, Shucheng Yin, Siran Zhang, Tingting Liu, Xianping Yin, Xiaoyu Yang, Xin Song, Xuan Hu, Yankai Zhang, and Yuqiao Li. Magi-1: Autoregressive video generation at scale, 2025.
- [3] Osayamen Jonathan Aimuyo, Byungsoo Oh, and Rachee Singh. Flashmoe: Fast distributed moe in a single kernel, 2025.
- [4] Quentin Anthony, Benjamin Michalowicz, Jacob Hatef, Lang Xu, Mustafa Abduljabbar, Aamir Shafi, Hari Subramoni, and Dhabaleswar Panda. Demystifying the communication characteristics for distributed transformer models, 2024.
- [5] Muhammad Awad, Muhammad Osama, and Brandon Potter. Iris: First-class multi-gpu programming experience in triton, 2025.
- [6] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-turn rl for generating cuda kernels, 2025.
- [7] Rohil Bhargava, Taylor Allison, and Harry Petty. Nvidia vera rubin pod: Seven chips, five rack-scale systems, one ai supercomputer, Apr 2026.
- [8] Boris Bonev, Thorsten Kurth, Christian Hundt, Jaideep Pathak, Maximilian Baust, Karthik Kashinath, and Anima Anandkumar. Spherical fourier neural operators: Learning stable dynamics on the sphere, 2023.
- [9] David Cardwell and Fengguang Song. An extended roofline model with communication-awareness for distributed-memory hpc systems. In *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region*, HPCAsia '19, page 26–35, New York, NY, USA, 2019. Association for Computing Machinery.
- [10] Nicolas Carion, Laura Gustafson, Yuan-Ting Hu, Shoubhik Debnath, Ronghang Hu, Didac Suris, Chaitanya Ryali, Kalyan Vasudev Alwala, Haitham Khedr, Andrew Huang, Jie Lei, Tengyu Ma, Baishan Guo, Arpit Kalla, Markus Marks, Joseph Greer, Meng Wang, Peize Sun, Roman Rädle, Triantafyllos Afouras, Effrosyni Mavroudi, Katherine Xu, Tsung-Han Wu, Yu Zhou, Liliane Momeni, Rishi Hazra, Shuangrui Ding, Sagar Vaze, Francois Porcher, Feng Li, Siyuan Li, Aishwarya Kamath, Ho Kei Cheng, Piotr Dollár, Nikhila Ravi, Kate Saenko, Pengchuan Zhang, and Christoph Feichtenhofer. Sam 3: Segment anything with concepts, 2025.
- [11] Fabio Checconi, Jesmin Jahan Tithi, and Fabrizio Petrini. Ridgeline: A 2d roofline model for distributed systems, 2025.
- [12] Jacek Czaja, Michal Gallus, Joanna Wozna, Adam Grygielski, and Luo Tao. Applying the roofline model for deep learning performance optimizations, 2020.

- [13] Weinan Dai, Hanlin Wu, Qiyang Yu, Huan ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, Wei-Ying Ma, Ya-Qin Zhang, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. Cuda agent: Large-scale agentic rl for high-performance cuda kernel generation, 2026.
- [14] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. Deepseek-v3 technical report, 2025.
- [15] Jiangfei Duan, Shuo Zhang, Zerui Wang, Lijuan Jiang, Wenwen Qu, Qinghao Hu, Guoteng Wang, Qizhen Weng, Hang Yan, Xingcheng Zhang, Xipeng Qiu, Dahua Lin, Yonggang Wen, Xin Jin, Tianwei Zhang, and Peng Sun. Efficient training of large language models on distributed infrastructures: A survey, 2024.
- [16] Jared Fernandez, Luca Wehrstedt, Leonid Shamis, Mostafa Elhoushi, Kalyan Saladi, Yonatan Bisk, Emma Strubell, and Jacob Kahn. Hardware scaling trends and diminishing returns in large-scale distributed training, 2025.
- [17] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [18] GLM-5-Team, :, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang, Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang,

Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia’ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhan Shen, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. *Glm-5: from vibe coding to agentic engineering*, 2026.

- [19] Raja Gond, Nipun Kwatra, and Ramachandran Ramjee. *Tokenweave: Efficient compute-communication overlap for distributed llm inference*, 2025.
- [20] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Allonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu, Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yeary, Laurens van der Maaten, Lawrence Chen,

Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kambadur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur Çelebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shaoliang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collot, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenyin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuwei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwon Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippas Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damla, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt,

Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Rutu Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta, Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojuan Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. The llama 3 herd of models, 2024.

- [21] Bodun Hu, Yoga Sri Varshan V, Saurabh Agarwal, and Aditya Akella. Cuco: An agentic framework for compute and communication co-design, 2026.
- [22] Zhiyi Hu, Siyuan Shen, Tommaso Bonato, Sylvain Jeaugey, Cedell Alexander, Eric Spada, James Dinan, Jeff Hammond, and Torsten Hoefer. Demystifying nccl: An in-depth analysis of gpu communication protocols and algorithms, 2026.
- [23] Gabriel Ilharco, Mitchell Wortsman, Ross Wightman, Cade Gordon, Nicholas Carlini, Rohan Taori, Achal Dave, Vaishaal Shankar, Hongseok Namkoong, John Miller, Hannaneh Hajishirzi, Ali Farhadi, and Ludwig Schmidt. Openclip, July 2021. If you use this software, please cite it as below.
- [24] Dmytro Ivchenko, Dennis Van Der Staay, Colin Taylor, Xing Liu, Will Feng, Rahul Kindi, Anirudh Sudarshan, and Shahin Sefati. Torchrec: a pytorch domain library for recommendation systems. In *Proceedings of the 16th ACM Conference on Recommender Systems*, RecSys '22, page 482–483, New York, NY, USA, 2022. Association for Computing Machinery.
- [25] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models, 2023.

- [26] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. Megascala: Scaling large language model training to more than 10,000 gpus, 2024.
- [27] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings, 2023.
- [28] Standard Kernel. Reimagining kernel generation at the ptx layer: An llm system learning from dsks to outperform them. Standard Kernel Blog, April 2026. Accessed: 2026-05-02.
- [29] Ronny Krashinsky, Olivier Giroux, Stephen Jones, Nick Stam, and Sridhar Ramaswamy. Nvidia ampere architecture in-depth, May 2023.
- [30] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards robust agentic cuda kernel benchmarking, verification, and optimization, 2025.
- [31] Muyang Li, Tianle Cai, Jiaxin Cao, Qinsheng Zhang, Han Cai, Junjie Bai, Yangqing Jia, Ming-Yu Liu, Kai Li, and Song Han. DistriFusion: Distributed parallel inference for high-resolution diffusion models, 2024.
- [32] Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, Qijing Huang, Zhifan Ye, Nestor Qin, Fengzhe Zhou, Yuan Zhang, Jingquan Wang, Sana Damani, Dheeraj Peri, Ouyue Xie, Aditya Kane, Moshe Maor, Michael Behar, Triston Cao, Rishabh Mehta, Vartika Singh, Vikram Sharma Mailthody, Terry Chen, Zihao Ye, Hanfeng Chen, Tianqi Chen, Vinod Grover, Wei Chen, Wei Liu, Eric Chung, Luis Ceze, Roger Bringmann, Cyril Zeller, Michael Lightstone, Christos Kozyrakis, and Humphrey Shi. Sol-execbench: Speed-of-light benchmarking for real-world gpu kernels against hardware limits, 2026.
- [33] Jiashuo Lin, Xingbo Feng, Hanrui Qi, Yan Liu, Chenxi Ling, Bo Tang, Yi Wang, Xiaofeng Tao, and Weichao Li. Flux: Fine-grained communication scheduling for distributed training in multi-tenant ai clusters. In *2025 IEEE/ACM 33rd International Symposium on Quality of Service (IWQoS)*, pages 1–2, 2025.
- [34] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context, 2023.
- [35] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Shaowei Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for llm training, 2025.
- [36] Mitrasish. Nvidia rubin r100: Specs, architecture, and gpu cloud availability: Spheron blog, Apr 2026.
- [37] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan

- Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm, 2021.
- [38] NVIDIA. cupynumeric. <https://github.com/nv-legate/cupynumeric>. GitHub repository, accessed 2026-05-06.
 - [39] NVIDIA. Nvidia tensorrt-llm. <https://github.com/NVIDIA/TensorRT-LLM>. GitHub repository, accessed 2026-05-06.
 - [40] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2024.
 - [41] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient gpu kernels?, 2025.
 - [42] PhysicsNeMo Contributors. Nvidia physicsnemo: An open-source framework for physics-based deep learning in science and engineering. <https://github.com/NVIDIA/physicsnemo>, 2023. GitHub repository.
 - [43] Michael Poli, Stefano Massaroli, Eric Nguyen, Daniel Y Fu, Tri Dao, Stephen Baccus, Yoshua Bengio, Stefano Ermon, and Christopher Ré. Hyena hierarchy: Towards larger convolutional language models. In *International Conference on Machine Learning*, pages 28043–28078. PMLR, 2023.
 - [44] PyTorch Team. Pytorch symmetric memory. https://docs.pytorch.org/docs/stable/symmetric_memory.html, 2026. Accessed: 2026-04-27.
 - [45] Gabin Schieffer, Jacob Wahlgren, Ruimin Shi, Edgar A Leon, Roger Pearce, Maya Gokhale, and Ivy Peng. Inter-apu communication on amd mi300a systems via infinity fabric: A deep dive. In *Proceedings of the International Symposium on Memory Systems*, MemSys ’25, page 238–250, New York, NY, USA, 2026. Association for Computing Machinery.
 - [46] Stanford CS336. Cs336 spring 2026 assignment 2: Systems. <https://github.com/stanford-cs336/assignment2-systems>, 2026. GitHub repository.
 - [47] Stuart Sul and Chris Re. Thunderkittens 2.0: Even faster kernels for your gpus, Feb 2026.
 - [48] Stuart H. Sul, Simran Arora, Benjamin F. Spector, and Christopher Ré. Parallelkittens: Systematic and practical simplification of multi-gpu ai kernels, 2025.
 - [49] Han Wang, Linfeng Zhang, Jiequn Han, and Weinan E. Deepmd-kit: A deep learning package for many-body potential energy representation and molecular dynamics. *Computer Physics Communications*, 228:178–184, 2018.
 - [50] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, and Zheng Zhang. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*, 2019.

- [51] Zhongzhen Wen, Yinghui Zhang, Zhong Li, Zhongxin Liu, Linna Xie, and Tian Zhang. Multikernelbench: A multi-platform benchmark for kernel generation, 2025.
- [52] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM*, 52(4):65–76, April 2009.
- [53] Jihao Xin, Marco Canini, Peter Richtárik, and Samuel Horváth. Global-qsgd: Allreduce-compatible quantization for distributed learning with theoretical guarantees. In *Proceedings of the 5th Workshop on Machine Learning and Systems*, EuroMLSys ’25, page 216–229, New York, NY, USA, 2025. Association for Computing Machinery.
- [54] Shanli Xing, Yiyang Zhai, Alexander Jiang, Yixin Dong, Yong Wu, Zihao Ye, Charlie Ruan, Yingyi Huang, Yineng Zhang, Liangsheng Yin, Aksara Bayyapu, Luis Ceze, and Tianqi Chen. Flashinfer-bench: Building the virtuous cycle for ai-driven llm systems, 2026.
- [55] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik R Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [56] Songlin Yang and Yu Zhang. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, January 2024.
- [57] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for gaussian splatting, 2024.
- [58] yifuwang, Horace He, and Luca Wehrstedt. Pytorch symmetricmemory: Harnessing nvlink programmability with ease, Feb 2025.
- [59] Chenggang Zhao, Shangyan Zhou, Liyue Zhang, Chengqi Deng, Zhean Xu, Yuxuan Liu, Kuai Yu, Jiashi Li, and Liang Zhao. DeepEP: an efficient expert-parallel communication library. <https://github.com/deepseek-ai/DeepEP>, 2025.
- [60] Da Zheng, Xiang Song, Qi Zhu, Jian Zhang, Theodore Vasiloudis, Runjie Ma, Houyu Zhang, Zichen Wang, Soji Adeshina, Israt Nisa, Alejandro Mottini, Qingjun Cui, Huzefa Rangwala, Belinda Zeng, Christos Faloutsos, and George Karypis. Graphstorm: All-in-one graph machine learning framework for industry applications. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD ’24, page 6356–6367, 2024.
- [61] Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun Liang, Jidong Zhai, and Xin Liu. Triton-distributed: Programming overlapping kernels on distributed ai systems with the triton compiler, 2025.
- [62] Zangwei Zheng, Xiangyu Peng, Tianji Yang, Chenhui Shen, Shenggui Li, Hongxin Liu, Yukun Zhou, Tianyi Li, and Yang You. Open-sora: Democratizing efficient video production for all, 2024.

A Prompt & In-Context Example

To prompt frontier models, we provide a single in-context example: a symmetric-memory element-wise add. This was chosen to be the simplest possible kernel that demonstrates the two primitives PKB problems require: the symmetric heap API for allocating peer-addressable buffers, and direct NVLink P2P data transfer via device pointers. All other problem-solving is left to the model.

```
1 import torch
2 import torch.distributed as dist
3 import torch.distributed._symmetric_memory as symm_mem
4 from utils.cuda_helpers import compile_cuda_extension
5
6 CUDA_SRC = r'''
7 #include <torch/extension.h>
8 #include <ATen/cuda/CUDAContext.h>
9 #include <cuda_runtime.h>
10
11 __global__ void symmetric_add_kernel(
12     const float* __restrict__ local_data,
13     const float* __restrict__ remote_data,
14     float* __restrict__ out,
15     int64_t n
16 ) {
17     int64_t idx = (int64_t)blockIdx.x * blockDim.x + threadIdx.x;
18     if (idx < n) {
19         out[idx] = local_data[idx] + remote_data[idx];
20     }
21 }
22
23 void symmetric_add_f32(
24     torch::Tensor local,
25     int64_t remote_ptr,
26     torch::Tensor out,
27     int64_t n
28 ) {
29     const int threads = 256;
30     const int blocks = (int)((n + threads - 1) / threads);
31     cudaStream_t stream = at::cuda::getCurrentCUDAStream().stream();
32     const float* remote = reinterpret_cast<const float*>(static_cast<uintptr_t>(
33 ↪ remote_ptr));
34
35     symmetric_add_kernel<<<blocks, threads, 0, stream>>>(
36         local.data_ptr<float>(),
37         remote,
38         out.data_ptr<float>(),
39         n
40     );
41     C10_CUDA_KERNEL_LAUNCH_CHECK();
42 }
43
44 PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
45     m.def("symmetric_add_f32", &symmetric_add_f32,
46         "UVA symmetric add: local + remote float buffers");
47 }
48 '''
49 _ext = None
50
```

```

51
52 def _get_ext():
53     global _ext
54     if _ext is None:
55         _ext = compile_cuda_extension("symmetric_add_uva_ext", CUDA_SRC)
56     return _ext
57
58
59 _symm_cache = None
60
61
62 def _get_symm_state(n: int, dtype: torch.dtype, device: torch.device):
63     global _symm_cache
64     if _symm_cache is not None:
65         c = _symm_cache
66         if c["n"] == n and c["dtype"] == dtype:
67             return c["buf"], c["hdl"], c["out"]
68
69     buf = symm_mem.empty(n, device=device, dtype=dtype)
70     hdl = symm_mem.rendezvous(buf, dist.group.WORLD)
71     out = torch.empty(n, device=device, dtype=dtype)
72     _symm_cache = {"n": n, "dtype": dtype, "buf": buf, "hdl": hdl, "out": out}
73     return buf, hdl, out
74
75
76 @torch.no_grad()
77 def solution(tensor: torch.Tensor) -> torch.Tensor:
78     assert tensor.is_cuda and tensor.is_contiguous()
79     assert tensor.dtype == torch.float32
80     assert dist.is_initialized()
81     assert dist.get_world_size() == 2
82
83     rank = dist.get_rank()
84     peer = 1 - rank
85     n = tensor.numel()
86
87     if rank == 0:
88         _get_ext()
89         dist.barrier()
90
91     buf, hdl, out = _get_symm_state(n, tensor.dtype, tensor.device)
92     buf.copy_(tensor.reshape(-1))
93     hdl.barrier(channel=0)
94
95     remote_ptr = int(hdl.buffer_ptrs[peer])
96     _get_ext().symmetric_add_f32(buf, remote_ptr, out, n)
97
98     return out.reshape_as(tensor)

```

Listing 1: Element-wise vector add across two ranks via symmetric memory and using CUDA’s unified virtual addressing capabilities. World size must be 2 in this example.

The following prompt structure was used, based on the KernelBench [41] repository. Note the sections marked in <> are dynamically filled in during evaluation.

```

1 """
2 You write custom CUDA operators to replace **high-level PyTorch and torch.
  ↳ distributed / NCCL-heavy paths** in the given architecture. The reference is
  ↳ a **correctness and API specification**, not a mandate to keep the same

```



```

    ↪ operator mix: re-express the workload in the **appropriate backend** (CUDA
    ↪ operators) wherever that unlocks speed.
3
4 **Minimize stock PyTorch on the hot path.** Avoid treating off-the-shelf 'torch.nn
    ↪ ' modules, generic elementwise/tensor ops, or opaque 'torch.distributed'
    ↪ collectives as the default implementation when the bottleneck is
    ↪ communication or fused math. Prefer explicit custom kernels, hand-managed
    ↪ buffers, and synchronization you control - only use PyTorch where the
    ↪ harness or interop requires it (e.g., process group setup, tensor shells,
    ↪ loading a compiled extension).
5
6 **Prioritize device-side communication.** Favor peer data movement and reductions
    ↪ expressed **on device** (symmetric memory, UVA / P2P access, cooperative
    ↪ patterns, CUDA IPC, or backend-native device APIs) instead of repeated host-
    ↪ driven launches of black-box collectives when a tighter device-resident or
    ↪ pipelined pattern fits the problem.
7
8 **Maximize compute communication overlap.** Reason about the schedule: chunking,
    ↪ double buffering, multiple streams, pipeline stages, and fused kernels so
    ↪ communication hides behind independent computation (or vice versa). Examples
    ↪ : pipelined allgather with matmul tiles, chunked reducescatter fused with
    ↪ the next projection, overlapping broadcast with compute on unrelated chunks,
    ↪ ring/tree algorithms implemented where they reduce critical-path latency.
9
10 You may replace or fuse collectives, reshape algorithms (ring, tree, recursive
    ↪ halving doubling), or merge communication with adjacent ops. Bias heavily
    ↪ toward **custom CUDA operators**, **device side data movement**, and **
    ↪ overlap** not toward a minimal diff from the reference PyTorch.
11
12 Here are examples showing how to embed custom CUDA operators in PyTorch:
13
14 Example A: elementwise add (reference vs CUDA extension)
15 < Example demonstrating baseline with unoverlapped Torch + NCCL >
16
17
18 Optimized with CUDA operators:
19 < In-context example demonstrating output format with optimized CUDA >
20
21 You are given the following PyTorch code to optimize:
22
23 < Reference Torch code to optimize >
24
25 Note: The kernels should be optimized for BF16 (bfloat16) precision.
26
27 Here is some information about the underlying hardware that you should keep in
    ↪ mind.
28
29 The GPU that will run the kernel is NVIDIA H100 80GB SXM, Hopper architecture.
30
31 - Four GPUs per node; assume peers are NVLink-connected within the same domain.
    ↪ High bisection bandwidth on-node; prefer kernels that exploit tensor cores
    ↪ and async copies.
32
33 Here are some concepts about the GPU architecture that could be helpful:
34
35 - **Tensor cores**: Use where possible for FP16/BF16/TF32 dense math; fall back to
    ↪ CUDA cores where shapes do not map cleanly.
36 - **NVLink / P2P**: Device pointers from symmetric memory should be valid for
    ↪ direct loads/stores across peers in this profile.

```

```

37
38 Here are some best practices for writing kernels on GPU:
39
40 - Overlap communication with computation where the schedule allows (e.g., chunked
    ↪ collectives, double-buffering).
41 - Keep working sets in fast memory hierarchy; minimize cross-GPU traffic for hot
    ↪ loops.
42
43 Rewrite 'solution()' to get a real speedup by **implementing the workload in the
    ↪ target backend** and **shrinking reliance on stock PyTorch** on the
    ↪ performance-critical path (keep the same signature; must run on every rank
    ↪ like the reference). Use symmetric memory / explicit device buffers and
    ↪ custom compiled code as appropriate.
44
45 Output format: You may include a short strategy that states how you exploit device
    ↪ -side communication and compute-communication overlap - then put all
    ↪ implementation in ``python`` fenced blocks: complete 'solution()',
    ↪ supporting kernels, extension/JIT loading, imports, rendezvous/buffers, etc.
    ↪ No long essays, no tutorial prose, no standalone test harness - only brief
    ↪ strategy (optional) plus working code. If you omit the strategy, the code
    ↪ structure must still reflect overlap and device-side patterns clearly.
46
47 Produce the complete Python file contents for that path.
48 """

```

Listing 2: System Prompt with task/examples/hardware topology information. This is dynamically filled in during evaluation time from a template.

B Survey of GPU Communication Frameworks

Traditional multi-GPU programs separate compute and communication: collective operations (e.g., all-reduce, all-gather) are launched from the host via libraries like NCCL, while compute runs in separate kernels. Fusing these into a single kernel, where communication happens via direct memory loads and stores from within the kernel itself rather than host-launched collectives, is known to yield significant performance gains [21].

PKB is designed around this fused paradigm. Problems require models to move beyond host-side NCCL and instead exploit device-side communication APIs, which collectively enable intra-SM communication: the Torch symmetric heap API for peer-addressable buffers accessible directly from CUDA kernels, NVLink P2P loads and stores via UVA pointers, and NVSwitch multicast via `multimem` PTX instructions. Beyond CUDA, PKB also accepts solutions written in Triton (via the Torch Symmetric Memory API [44]) and ParallelKittens [48], both of which operate within the same intra-node NVLink domain. Whether higher levels of abstraction help or hurt model performance on these tasks is an interesting open question we leave to future work.

For completeness, we briefly survey the inter-node device-side communication landscape, which PKB does not currently cover. NVSHMEM is NVIDIA’s native library for inter- and intra-node GPU communication, offering RMA, collectives, atomics, and memory operations; its recent `nvshmem4py` interface and interoperability with Numba-CUDA, CuTe, and Triton have lowered its adoption barrier, though its memory safety features can impose performance overhead [48]. Triton Distributed similarly spans intra- and inter-node settings, though it primarily targets H800 GPUs. Most recently, NCCL GIN (introduced in NCCL 2.28) provides a device-side API enabling GPUs to directly initiate RDMA transfers without CPU involvement, offering a first-party path for inter-node fused kernels [22].

PKB is intentionally scoped to intra-node settings. This scope excludes certain problem classes that are more naturally inter-node in character — including prefill-decode disaggregation, certain pipeline parallelism patterns, and delta encoding — as these appear more frequently on slower network fabrics (e.g., InfiniBand, RoCE, EFA) where the device-side API landscape is less mature and hardware assumptions are harder to standardize. We view inter-node kernel generation as a natural and important extension of PKB, and welcome contributions in that direction.

C Problem Sources

Source repositories.

- NVIDIA/nccl [22]
- stanford-cs336/assignment2-systems [46]
- HazyResearch/ThunderKittens [47, 48]
- NVIDIA/Megatron-LM [37]
- deepspeedai/DeepSpeed [25]
- sands-lab/Global-QSGD [53]
- deepseek-ai/DeepEP [59]
- NVIDIA/TensorRT-LLM [39]
- zhuzilin/ring-flash-attention [34]
- mlfoundations/open_clip [23]
- NVIDIA/physicsnemo [42]
- nerfstudio-project/gspat [57]
- NVIDIA/torch-harmonics [8]
- deepmodeling/deepmd-kit [49]
- pyg-team/pytorch_geometric [17]
- dmlc/dgl [50]
- awslabs/graphstorm [60]
- pytorch/torchrec [24]
- Zymrael/savanna [43]
- fla-org/flash-linear-attention [56]
- hpcaitech/Open-Sora [62]
- SandAI-org/MAGI-1 [2]
- facebookresearch/dinov2 [40]
- facebookresearch/sam3 [10]
- NVIDIA-NeMo/RL [1]
- NVIDIA-NeMo/emerging-optimizers [35]
- nv-legiate/cupynumeric [38]
- mit-han-lab/distrifuser [31]

Tier	Count	Description
Bare collective primitives	8	Fundamental communication operations without any added computation. These serve as a baseline for the raw data-transfer speed of the system.
Transition kernels	8	Simple mathematical operations—like RMSNorm or data compression (FP8 quantization)—that are fused directly into communication steps to reduce overhead.
Layer fragments	18	Building blocks of neural network layers, such as GEMM or convolutions, that have been split across multiple GPUs using standard parallelism strategies.
End-to-end composites	16	Complex, multi-step operations that coordinate both calculation and data movement, such as full Ring Attention or MoE routing and dispatch.
Optimizer / update kernels	9	Operations used during the model update phase, where gradients and parameters are synchronized and updated across a cluster.
Distributed loss functions	5	Final-stage calculations that compare model outputs to targets across all GPUs, such as cross-entropy for very large vocabularies or contrastive losses for vision models.
Application-specific kernels	20	Specialized workloads tailored for non-LLM domains like GNNs, Scientific simulations, and 3D Graphics, as detailed in Table 5.

Table 4: Abstraction tiers covered by PKB.

Methodology. We selected the 87 PKB problems by starting from the parallelism strategies used in production training and inference systems, then choosing representative kernels from the source repositories listed above. For LLM-specific workloads, the majority of the benchmark, production code usually breaks down into composable sharding strategies (tensor, expert, sequence/Ulysses, context/ring, data, FSDP) applied to attention and feedforward layers, with normalization between stages. Each path through this taxonomy (Figure 1) creates optimization opportunities both inside layers and at the transition points where data moves between parallel layouts. These boundary kernels are especially important because communication is not just a separate collective. It is part of the operation itself, like in MoE routing, vocab-parallel log-prob computation, and ring attention. Thus, PKB includes many variable-size and irregular communication patterns. Beyond LLMs, we include 22 problems from other domains, summarized in Table 5. We also group all PKB problems into six abstraction tiers in Table 4: the first tier is bare collectives, and the other tiers combine computation and communication at higher levels of complexity.

Coverage beyond standard LLM kernels. Figure 3 summarizes coverage by parallelism axis and domain; Table 5 describes what the non-LLM problems contribute that standard transformer problems don’t. We under-emphasize pipeline parallelism (1 problem) since PP performance is in practice mostly inter-node and governed by scheduling, and we exclude prefill/decode disaggregation as primarily an inference-serving scheduling concern (not a kernel concern).

Block	Problems	What it contributes
GNN	65–70	Graph workloads present ownership-based routing: each rank owns different nodes or features, so data must be fetched, pushed, duplicated, or scored across GPUs. This creates variable-size all-to-all communication and load imbalance from uneven graph neighborhoods, making GNN parallelism closer to sparse routing than dense tensor parallelism.
Scientific	60–64	Scientific kernels add non-transformer communication patterns: distributed FFT transposes, spherical convolutions over a spatial process grid, shared reductions inside an optimizer, and 3D Gaussian splatting where cameras and projected points must be redistributed. These problems stress emphasize layouts rather than token or expert layouts.
Vision & diffusion	59, 77–82, 88	Vision and diffusion models introduce communication over images, tiles, masks, and feature databases. Examples include ring contrastive losses, async attention with all-to-all exchange, tile-parallel VAE decoding, distributed k-NN, Sinkhorn assignment, mask-IoU suppression, and Conv2d boundary exchange.
Recommendation / sparse	10, 71	These workloads deal with "jagged" data (like variable-length lists of user interests). Because the data is not in neat, equal-sized blocks, the system is forced to move lists of varying lengths, which is a major stress test for data routing.

Table 5: Non-LLM problem categories and what they contribute.

D Communication-Aware Roofline

The classical roofline bounds runtime by peak compute P_{peak} and HBM bandwidth B_{mem} [52]. For distributed kernels, we add NVLink bandwidth B_{comm} as a third ceiling, since it is often the bottleneck. A naive extension takes the three-way maximum across all three resources, but this overstates the achievable speedup whenever a workload contains a barrier that no implementation can hide. We therefore use a phase-aware roofline where communication is a third resource and we serialize only at unavoidable dependencies.

D.1 Phases as dependency boundaries

Fused kernels can overlap compute, HBM traffic, and communication (e.g. a fast GEMM–all-gather implementation streams weights into the matmul units while later tiles are still in flight on the network). We model these patterns with a per-resource max. However, some operations enforce hard dependencies: for example, an MoE routing all-to-all must finish before expert compute begins. We model these with a sum.

We decompose each problem into phases within which the three resources overlap freely. Within a phase, we group operations by tile-pipelining ability: anything that can stream from one operation’s output into another’s input at sub-tensor granularity belongs to the same phase, and anything requiring a fully formed intermediate sits across a boundary. We also report a coarse non-phase-aware roofline per problem.

D.2 The roofline equation

For problem i split into phases k with FLOPs W_k , HBM traffic $Q_{\text{mem},k}$, and communication volume $Q_{\text{comm},k}$, the phase-aware roofline time is

$$T_{\text{roof},i} = \sum_k \max\left(\frac{W_k}{P_{\text{peak}}}, \frac{Q_{\text{mem},k}}{B_{\text{mem}}}, \frac{Q_{\text{comm},k}}{B_{\text{comm}}}\right). \quad (1)$$

The three peaks P_{peak} , B_{mem} , B_{comm} come from the NVIDIA H100 SXM5 specs, and the three numerators come from a maximally fused implementation of each stage.

Compute. P_{peak} is set per stage by precision and execution path. For H100 SXM5 we use dense, non-sparse peaks: 989 TFLOP/s (BF16/FP16 Tensor Core), 1979 TFLOP/s (FP8 Tensor Core), 494 TFLOP/s (TF32 Tensor Core), 67 TFLOP/s (FP32 SM cores).

HBM. $B_{\text{mem}} = 3.35$ TB/s. $Q_{\text{mem},k}$ counts bytes crossing the L2–HBM boundary; register- and SRAM-resident intermediates are free.

Communication. $B_{\text{comm}} = 450$ GB/s per direction. $Q_{\text{comm},k}$ uses collective bus-volume formulas (e.g. $\frac{W-1}{W}M$ for a ring all-gather of M bytes across W ranks). When a phase spans multiple process-group axes (e.g. TP and CP), each stage is charged on its own axis, and times sharing a network resource are summed before the phase max.

D.3 Cost models

Per problem, we contribute a Python cost-model file that declares its phases, per-stage FLOP / HBM / communication formulas, and the symbolic shape variables they depend on (like world size, sequence length, expert count). To evaluate, we bind the shape variables to a hardware profile supplying P_{peak} , B_{mem} , and B_{comm} . The same model produces T_{roof} at any shape or world size.

D.4 Worked example: Problem 72 Hyena Conv1D Boundary Exchange

This problem is a depthwise 1D conv with zigzag context-parallel sharding. Each rank owns chunks of shape $[B, H, 2S]$ and filter $[H, 1, K]$, exchanges $K-1$ -element halos with both neighbors, then runs a local 1D conv. Halos stream from P2P receive buffers into registers while the convolution computes, so the workload is one phase with two stages:

- Halo P2P: $W = 0$, $Q_{\text{mem}} = 0$, $Q_{\text{comm}} = 2BH(K-1)d_b$.
- Depthwise Conv1D: $W = 4BHSK$, $Q_{\text{mem}} = (4BHS + HK)d_b$, $Q_{\text{comm}} = 0$.

At $(B, H, K, S, W, d_b) = (1, 1024, 7, 1024, 4, 2)$ the roofline gives $W/P_{\text{peak}} = 438$ ns, $Q_{\text{mem}}/B_{\text{mem}} = 2,508$ ns, and $Q_{\text{comm}}/B_{\text{comm}} = 55$ ns, so $T_{\text{roof},72} \approx 2.51 \mu\text{s}$ (memory-bound).

D.5 Baseline utilization across PKB

We classify each problem’s binding resource as whichever of compute, HBM, or NVLink contributes the largest share of $T_{\text{roof},i}$ summed across phases. Of the PKB problems, 75% are NVLink-bound, 18% are HBM-bound, and 7% are compute-bound, confirming that the benchmark is dominated by communication-limited workloads and motivating the third ceiling in Equation 1.

Stratified utilization. We compute the PyTorch reference utilization $U_i = \min(1, T_{\text{roof},i}/T_{\text{ref},i})$ on every task and stratify by binding resource:

Binding	N	% $U_{\text{ref}} < 0.5$
NVLink (communication)	65	100%
HBM (memory)	16	100%
Compute	6	83.3%

Table 6: PyTorch reference utilization, stratified by the binding resource of each problem’s roofline. Every PyTorch baseline on the 81 communication- and memory-bound tasks (93% of PKB) falls below half its hardware ceiling.

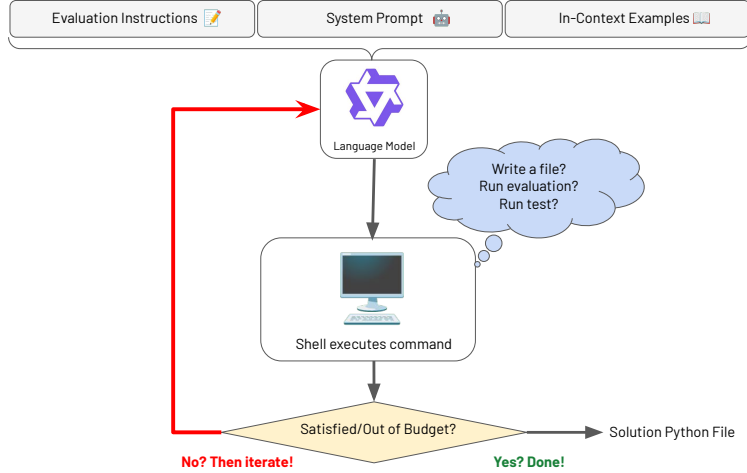


Figure 9: **Flow of agent harness.** The LLM can make arbitrary modifications to an existing codebase based on the output of the provided evaluation script. Test-time compute is set to a limit of 20 iterations, i.e. LLM calls. The harness used is mini-swe-agent [55].

E Agentic Harness

In addition to single-shot prompting (Appendix A), we evaluate models in an agentic setting using mini-swe-agent [55]. The agent is given the same kernel specification as the single-shot setting, but instead of producing a solution in one pass, it operates in a loop: the LLM observes its current context, decides which shell command to run (e.g., writing code to a file, executing the evaluation script, reading error output), and iterates based on the result. Concretely, the agent is provided with an evaluation command it can invoke at any point to run its current solution against the correctness and performance harness on real hardware — giving it the same feedback signal a human developer would have. The loop terminates when the agent is satisfied with its solution or a turn budget is exhausted. An illustration is provided in Figure 9. The agent was able to correctly solve 31 problems, with 26 solutions being faster than the reference.

F Kernels of Interest

Here, we highlight a selection of generated kernels that exhibit effective multi-GPU optimization strategies. We highlight kernels that outperform strong baselines, as well as those that represent genuinely novel contributions as of writing.

F.1 NCCL Collectives (Gemini 3 Pro)

We observe several generated kernels outperform NCCL collectives outright at low tensor sizes. Gemini 3 Pro (single shot) produced an AllGather kernel running $1.2\times$ faster than the NCCL baseline on tensor shape (1024×1024) , and a Scatter kernel running $1.626\times$ faster on Rank 0 tensor shape $4 \times 1024 \times 1024$ (world size is 4). Performance matches NCCL at smaller matrix sizes (Figure 10), but degrades noticeably as matrix size increases. One key optimization the model independently identified was dynamically selecting the widest available memory access width based on buffer alignment at runtime (Figure 11).

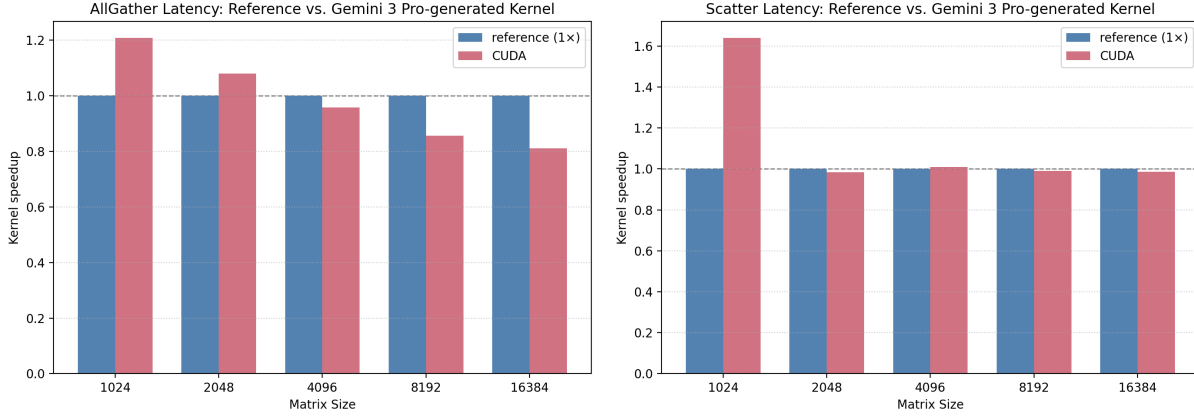


Figure 10: **Generated symmetric-memory kernels can approach NCCL performance on Scatter and AllGather collectives.** "Reference" in this case refers to the torch.distributed NCCL collective. The model evaluated is Gemini 3 Pro (Single-Shot).

```

1 // Dynamically use the widest memory instructions possible based on alignment
2 if (chunk_bytes % 16 == 0) {
3     int64_t chunk_16 = chunk_bytes / 16;
4     const uint4* src_16 = reinterpret_cast<const uint4*>(src);
5     uint4* dst_16 = reinterpret_cast<uint4*>(dst_ptr);
6     for (int64_t i = idx; i < chunk_16; i += stride) {
7         dst_16[i] = src_16[i];
8     }
9 } else if (chunk_bytes % 8 == 0) {
10     int64_t chunk_8 = chunk_bytes / 8;
11     const uint2* src_8 = reinterpret_cast<const uint2*>(src);
12     uint2* dst_8 = reinterpret_cast<uint2*>(dst_ptr);
13     for (int64_t i = idx; i < chunk_8; i += stride) {
14         dst_8[i] = src_8[i];
15     }
16 }

```

Figure 11: **Gemini 3 Pro Key Optimization: vectorized memory copy kernel.** To optimize an NCCL AllGather operation, the model dynamically selects the widest available memory instruction based on buffer alignment: 128-bit uint4 loads/stores when the chunk size is 16-byte aligned, falling back to 64-bit uint2 otherwise. This maximizes memory throughput on the GPU.

F.2 Context Parallel: Ulysses (Gemini 3 Pro)

Sequence parallelism via Ulysses Attention redistributes a tensor from a sequence-parallel to a head-parallel layout before the attention computation, then back again afterward [25]. In the reference implementation, this is performed as a gather over the sequence dimension followed by a scatter over the head dimension, both via `all_to_all` calls interleaved with reshapes, transposes, and concatenations, all of which induce heavy framework overhead.

Models perform surprisingly well on this class of problems, averaging a 28.3% solve rate in the PKB testing harness (Table 2). The best generation (produced by Gemini 3 Pro) achieves a $\sim 3.73\times$ speedup over the PyTorch + NCCL baseline on the `GatherHeadsScatterSeqQKV` kernel at a local sequence length per rank of $T = 1024$. The input tensor is configured with a shape of $(B, T, 3 \times H \times d) = (1, 1024, 3072)$, where $B = 1$ is the batch size, $H = 16$ is the number of attention heads, and $d = 64$ is the per-head dimension, packed as a single QKV projection tensor. Speedups across local sequence lengths are shown in Figure 12.

The key insight Gemini 3 Pro identified is that the entire collective communication block can be recast as a direct NVLink *pull* kernel over symmetric memory (Figure 13), eliminating host-orchestrated data movement entirely. The reference implementation incurs multiple sequential memory passes (split, all-to-all, concat, reshape), whereas the generated kernel fuses these into two operations per rank: a single contiguous write of local data into a symmetric buffer, followed by a single NVLink read in which each rank directly pulls the sharded slices it needs from its peers into the target destination layout. Destination offsets are computed analytically via index decomposition inside the kernel threads, completely eliminating intermediate allocations and host-side synchronization points.

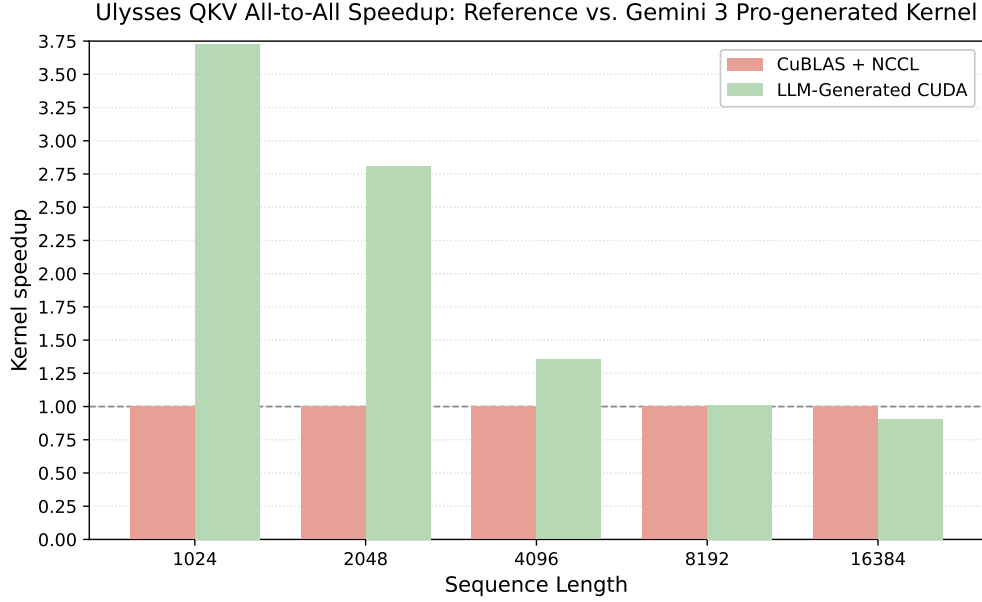


Figure 12: **LLM-generated DeepSpeed-Ulysses QKV All-to-All kernel speedup over reference.** Performance improvement of the optimized CUDA kernel relative to the CuBLAS + NCCL baseline ($1\times$) across a sweep of local sequence lengths T per GPU rank. Evaluations use a fixed batch size $B = 1$, 16 attention heads, and a head dimension of $d = 64$, packed into a single QKV projection tensor. The custom symmetric-memory kernel achieves a peak speedup of $\sim 3.73\times$ at a local sequence length of $T = 1024$ and remains competitive across the entire sweep before converging with the reference at $T = 8192$ as bulk data transfer volume scales.

```

1 __global__ void ulysses_pull_kernel(
2     const uint64_t* __restrict__ peer_ptrs,
3     void* __restrict__ out_ptr,
4     int64_t A, int64_t B, int64_t C, int64_t D, int64_t E_vec,
5     int P, int my_rank, bool scatter_first, int64_t N_out_vec
6 ) {
7     int64_t idx = (int64_t)blockIdx.x * blockDim.x + threadIdx.x;
8     if (idx >= N_out_vec) return;
9
10    int64_t e = idx % E_vec;
11    int64_t tmp = idx / E_vec;
12
13    int64_t rank_j;
14    int64_t in_idx;
15
16    if (scatter_first) {
17        // B is scatter (head_dim), D is gather (seq_dim)
18        // Output shape: [A, B/P, C, D*P, E_vec]
19        int64_t d_out = tmp % (D * P);
20        tmp = tmp / (D * P);
21        int64_t c = tmp % C;
22        tmp = tmp / C;
23        int64_t b_out = tmp % (B / P);
24        int64_t a = tmp / (B / P);
25
26        rank_j = d_out / D;
27        int64_t d_in = d_out % D;
28        int64_t b_in = my_rank * (B / P) + b_out;
29
30        in_idx = (((a * B + b_in) * C + c) * D + d_in) * E_vec + e;
31    }
32 }

```

Figure 13: NVLink pull kernel generated by Gemini 3 Pro. Rather than staging data through intermediate buffers as in a conventional AllToAll, each thread directly computes its source peer and remote index, then pulls the required element over NVLink. The `scatter_first` flag controls whether the scatter dimension is the head dimension D or the sequence dimension B , handling both Ulysses gather-scatter orderings in a single kernel and eliminating all packing and staging overhead.

F.3 NeMo Vocab-Parallel LogP with Top-k/Top-p Filtering (Gemini 3 Pro)

We identified a distributed operation in NVIDIA NeMo-RL [1] that computes per-token log-probabilities of target tokens under a top-k/top-p filtered distribution, where logits are sharded across the vocabulary dimension over a tensor-parallel group. The reference uses `all_to_all` to transpose the layout from vocab-parallel to sequence-parallel, applies top-k/top-p filtering, computes `F.log_softmax`, gathers the target log-probabilities, and finally performs `all_gather` on the scalar results across ranks.

An illustrative comparison of the reference and generated solutions is provided in Figure 15. We find that the optimized solution generated by Gemini 3 Pro is $> 1.2\times$ faster at a local sequence length of $l = 1024$ on the input configuration (batch size $B = 1$, global vocabulary size $V = 32,000$, corresponding to 8,000 local vocabulary entries per rank distributed across 4 ranks) with top- $k = 10$ and top- $p = 0.9$. Speedups across different sequence lengths are shown in Figure 14.

The kernel derives its speedup from two sources; first, it replaces `all_to_all` with a fine-grained communication kernel (`push_logits_kernel_vec`), utilizing vectorized `uint4`/128-bit stores to stream logits directly into peer symmetric memory while computing layout permutations implicitly in index arithmetic to eliminate NCCL staging overhead. Second, it fuses log-softmax, target token extraction, and the final all-gather into a single block-per-token kernel (`fused_log_softmax_gather_push`) that executes two-pass warp-shuffle reductions in shared memory and directly populates all peer scalar log-probability buffers in one shot.

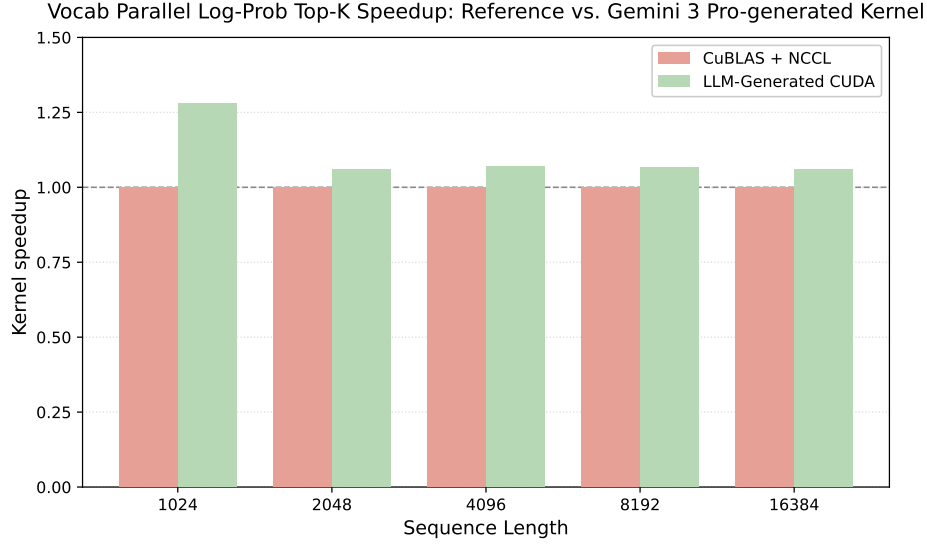


Figure 14: **LLM-generated VocabP LogProb TopK kernel speedup over reference.** Speedup of the optimized CUDA kernel (detailed in Figure 15) over the baseline implementation across a sweep of sequence lengths M within the PKB testing harness [1]. All evaluations are executed across 4 ranks with a fixed workload configuration: batch size $B = 1$, a global vocabulary size of $V = 32,000$ (8,000 local vocabulary entries per rank), top- $k = 10$, and top- $p = 0.9$. Correctness and speedup is measured over 100 benchmarking trials.

PyTorch Reference:

```

1 # [1] Transpose layout:
2 #   vocab-parallel -> seq-parallel
3 logits = all_to_all_single(logits_2d)
4 # [2] Filter + score target token
5 logits = apply_top_k_top_p(logits)
6 log_probs = F.log_softmax(logits)
7 token_lp = torch.gather(
8     log_probs, target)
9 # [3] Replicate scalar log-probs
10 dist.all_gather(gathered, token_lp)

```

LLM-Generated Solution:

```

1 for start in range(0, num_tokens,
2     chunk_tokens):
3     # Replaces [1]: vectorized peer push
4     # uint4 stores + implicit permute
5     __global__ push_logits_kernel_vec():
6         dst[t_local * W*V + rank * V]
7         = src_vec[i] // 128-bit store
8     recv_hdl.barrier()
9     # [2] unchanged: top-k/p filter
10    filtered = apply_top_k_top_p(...)
11    # Replaces [1]+[3]: fused kernel
12    __global__ fused_log_softmax_
13        gather_push():
14        // warp-shuffle max+sum reduction
15        lp = target_val - (max+log(sum))
16        for dst in range(world_size):
17            peer_buf[dst][token] = lp
18    logprobs_hdl.barrier()

```

Figure 15: **Gemini 3 Pro can optimize a vocabulary-parallel log-probability kernel via symmetric memory and fused reductions.** The reference transposes logits from vocab-parallel to sequence-parallel layout via `all_to_all`, applies top-k/top-p filtering, computes `log_softmax`, and replicates scalar results via `all_gather`. The LLM-generated solution replaces both collectives: a vectorized push kernel writes logits directly into peer symmetric memory buffers (with the layout permutation implicit in the index arithmetic), and a fused block-per-token kernel performs warp-shuffle softmax reduction and writes each scalar log-probability directly to all peer buffers, eliminating NCCL staging entirely.

F.4 Hyena Forward CP (GPT-5.5)

This kernel implements a critical section of the **Hyena context-parallel forward pass** from the official repository [43]. Each rank processes sequence-sharded activations x_1, x_2, v of shape $[B, D, l]$, where B is the batch size, D is the model hidden dimension, and $l = L/N$ is the local sequence length for a global sequence length L distributed across N GPUs.

To scale this architecture across a distributed cluster using Context Parallelism, the workload must handle two conflicting layout requirements. The upstream linear layers mix information across channels token-by-token, requiring a sequence-sharded layout where each rank holds a slice of time for all global channels D . Conversely, the Fast Fourier Transform (FFT) used to compute the global convolution requires the entire sequence length L for a single channel.

The baseline implementation resolves this by alternating tensor layouts via two communication round-trips using `all_to_all` collectives. The first round-trip scatters channels while gathering the full sequence length L , transitioning the cluster from sequence-parallel to channel-parallel so the local FFT convolution can execute independently on each rank’s channel shard. A second round-trip then transposes the data back into the original sequence-sharded layout, utilizing a specialized zigzag sequence indexing strategy to reorder the chunks during these communication passes.

The generated kernel achieves a $\sim 3.55\times$ speedup at a local sequence length of $l = 1024$ by replacing the two `all_to_all_single` round-trips with direct UVA peer reads/writes. This eliminates intermediate materialized tensors and fuses the gating, bias, and zigzag reindexing directly into the data-movement passes (Figure 17). However, as shown in Figure 16, this performance advantage does not hold at longer sequence lengths: the speedup decreases to $\sim 1.82\times$ at $l = 2048$ and falls to a performance penalty of $\sim 0.70\times$ at both $l = 8192$ and $l = 16,384$. This behavior highlights a fundamental hardware trade-off: while the custom kernel successfully slashes launch and allocation overheads for smaller payloads, highly optimized bulk transfers via NCCL collectives achieve superior NVLink bandwidth saturation compared to the fine-grained, thread-level UVA peer accesses used as the data transfer volume scales.

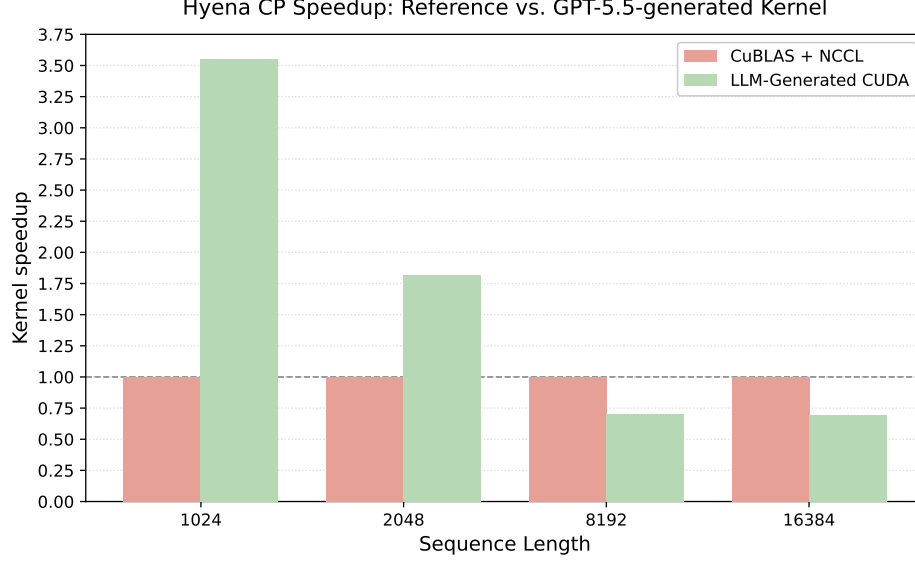


Figure 16: **LLM-generated Hyena CP kernel outperforms reference at short sequence lengths.** Speedup over the CuBLAS + NCCL reference implementation [43] across local sequence lengths from 1024 to 16,384 on 4 GPUs. The generated kernel achieves a peak speedup of $\sim 3.55\times$ at local sequence length $l = 1024$ by fusing data movement and operations, but drops below the reference ($\sim 0.70\times$) at larger sequence lengths ($l \geq 8192$) where bulk communication transfers provide more benefit than fine-grained peer-to-peer access. Correctness and speedup is measured over 100 benchmarking trials.

PyTorch Reference:

```

1 # [1] Seq-sharded -> channel-sharded
2 #   (all_to_all x3, one per tensor)
3 x1 = _a2a_split_to_full(x1_seq, ...)
4 x2 = _a2a_split_to_full(x2_seq, ...)
5 v = _a2a_split_to_full(v_seq, ...)
6 # [2] Local FFT convolution
7 z = x2 * v
8 z = _fftconv_ref(z, h_local,
9                 bias_local)
10 z = x1 * z
11 # [3] Channel-sharded -> seq-sharded
12 #   (all_to_all + zigzag reindex)
13 z_seq = _a2a_full_to_split(
14     z, group, with_zigzag)

```

LLM-Generated Solution:

```

1 # Replaces [1]: pack all three into
2 # one symmetric buffer, one barrier
3 pack3_bf16(x1, x2, v, inp_symm)
4 inp_hdl.barrier()
5 # Replaces [1]+[2] partially: UVA
6 # peer reads fused with x2*v and
7 # inverse zigzag reindex in one kernel
8 __global__ gather_x1_and_u():
9     pre_t = inv_zigzag_chunk(ch, W)
10    u[idx] = bf16(x2[peer]) * bf16(v[peer])
11    x1_full[idx] = x1[peer]
12 # [2] unchanged: FFT convolution
13 rfft -> complex_filter_mul -> irfft
14 # Replaces [2]+[3]: bias, x1*z,
15 # and scatter to [B,l,D] in one kernel
16 __global__ finalize_z + scatter_bs1():
17    conv = y[t] + u[t] * bias[c]
18    z_symm[t] = x1_full[t] * conv
19 z_hdl.barrier() // then UVA scatter

```

Figure 17: **GPT-5.5 eliminates redundant all-to-all collectives in Hyena context parallelism via fused symmetric-memory kernels.** The reference performs three separate `all_to_all` transposes (one per input tensor) to go from sequence-sharded to channel-sharded layout, runs the local FFT convolution, then transposes back. The LLM-generated solution packs all three inputs into a single symmetric allocation and replaces the first round of collectives with a single UVA peer-read kernel (`gather_x1_and_u`) that simultaneously computes $x_2 \cdot v$ and applies the inverse zigzag reindexing inline. The final `all_to_all` back is replaced by `scatter_final_bs1`, which fuses the residual bias, $x_1 \cdot z$ multiply, and transpose to $[B, l, D]$ into a single device-side scatter.

F.5 SAM3 all-gathered mask IoU suppression (GPT-5.5)

This kernel implements **SAM3 all-gathered mask IoU suppression** for distributed video object segmentation, adapted from the official SAM3 repository [10]. Each rank owns a variable number of tracked object masks and their object scores. The reference implementation uses two `dist.all_gather` calls (one for masks, one for scores), and computes pairwise IoU scores between all gathered masks in nested loops before applying a filter that removes masks above a fixed IoU threshold.

The generated kernel achieves a $> 1.25\times$ speedup at the baseline configuration of 16 tracked objects with 256×256 masks and an IoU threshold of 0.7. The speedup comes from collapsing this pipeline into four CUDA kernels backed by symmetric memory (Figure 19): each rank packs its local masks and scores into a symmetric buffer at the offset corresponding to its global object slice. However, as shown in Figure 18, this relative advantage diminishes as the total number of tracked objects scales up: the speedup tapers to $\sim 1.13\times$ at 64 objects and plateaus at $\sim 1.03\times$ at 512 objects. This indicates that while the custom symmetric-memory approach successfully slashes per-call overhead for fewer objects, the quadratic scaling of the IoU comparisons eventually dominates the runtime profile, causing performance to converge with the reference implementation.



Figure 18: **LLM-generated SAM3 IoU kernel exceeds reference at low** . Speedup of the generated CUDA kernel over the reference implementation [10] across a varying number of tracked objects at a fixed mask resolution of 256×256 . The generated kernel achieves a peak speedup of $> 1.25\times$ at 16 objects by minimizing per-call overhead. At larger object counts, performance diminishes as bulk data transfers begin to dominate the runtime profile. Correctness and speedup is measured over 100 benchmarking trials.

PyTorch Reference:

```

1 # [1] Two variable all-gathers
2 masks = _all_gather_variable(
3     masks_local, counts, group)
4 scores = _all_gather_variable(
5     scores_local, counts, group)
6 # [2] Dense IoU matmul on float masks
7 binary = masks > 0
8 flat = binary.flatten(1).float()
9 inter = flat @ flat.T # N x HW x N
10 area = flat.sum(dim=1)
11 union = area[:,None]+area[None,:]-inter
12 iou = inter / union.clamp_min(1.0)
13 # [3] Suppression via broadcast compare
14 overlaps = triu(iou >= thresh, 1)
15 suppress = (overlaps & cmp(last_i,
16     last_j) & (last_j > -1))...
17 masks[suppress] = _NO_OBJ_LOGIT

```

LLM-Generated Solution:

```

1 # Replaces [1]: pack into symmetric
2 # buffer at global offset, one barrier
3 pack_local_to_sym(masks, scores,
4     sym_flat, global_offset)
5 hdl.barrier()
6 # Replaces [1]+[2] partially: UVA peer
7 # reads fused with bitpack + area
8 __global__ gather_bitpack_area():
9     dst[p] = peer_base[obj * pixels + p]
10     bits |= (dst[p] > 0) << b // 32/word
11     area = popcount-reduce(bits)
12 # Replaces [2]+[3]: bitset IoU
13 __global__ pair_suppress_bitset():
14     for w in words:
15         inter += __popc(bi[w] & bj[w])
16         if inter >= thresh * union:
17             suppress[i or j] = 1
18 # Replaces in-place fill
19 __global__ apply_suppression():
20     if suppress[obj]:
21         masks_out[idx] = -10.0f

```

Figure 19: **LLM eliminates dense IoU matmul and dual all-gathers via bitpacked symmetric-memory kernels.** The reference performs two `dist.all_gather` calls and computes pairwise IoU as a dense matrix multiplication over float-binarized masks. The LLM-generated solution packs both masks and scores into a single symmetric buffer (replacing both all-gathers with one barrier), then a fused gather kernel reads peer memory over UVA pointers while simultaneously bitpacking each mask to 32-bit words and computing the per-object area.

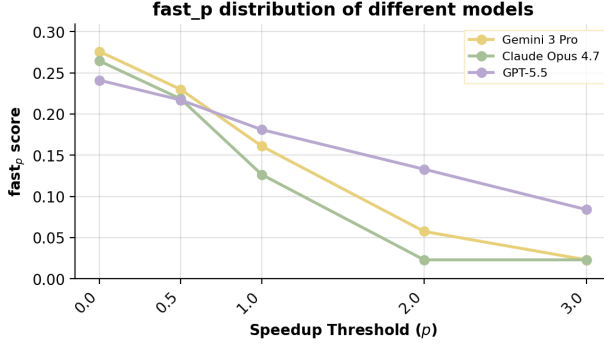


Figure 20: **Kernels remain slow on Blackwell.** Fast_p drops monotonically with speedup threshold p across all three evaluated models. Although GPT-5.5 degrades most gracefully, fewer than 20% of generated kernels outperform the unoverlapped baseline across all models ($\text{fast}_1 < 0.2$).

G Blackwell GPU Performance

We ran the same evaluation on Blackwell for three frontier models (Gemini 3 Pro, Claude Opus 4.7, and GPT-5.5) and find that kernels remain slow. This experiment uses 4 B200 nodes connected via 5th-generation NVLink/NVSwitch, with all other configurations matching the Hopper evaluation (Section 5). As shown in Figure 20, fast_p decreases sharply with the speedup threshold across all three models. Model correctness (fast_0 , i.e., the fraction of kernels that execute without error) remains below 30% across all models. These results confirm that switching to Blackwell does not resolve the kernel slowness observed on Hopper, suggesting that the bottleneck lies not in hardware generation but in the fundamental difficulty LLMs face in reasoning about multi-GPU parallelism.

H Future Work

PKB focuses on intra-node NVLink topologies and deliberately excludes problems where performance is governed by scheduling rather than kernel design, such as pipeline parallelism and prefill-decode disaggregation. Evaluations are conducted on H100 with NVSwitch; extending coverage to other topologies is left to future work. Future work may also leverage more specialized agentic systems and reasoning techniques to improve performance. PKB has natural support for ParallelKittens and Triton, and studying the impact of different programming abstractions — alongside emerging interfaces like cuTile’s upcoming networking features, NCCL GIN, and NVSHMEM — would be valuable follow-on work. Finally, extending the benchmark to internode communication via RoCE and InfiniBand is a natural next step, in addition to potentially expanding to other accelerators and topologies such as TPUs. We hope that our work inspires development and progress in this domain.